Exam Study Guide

Databricks Certified Data Engineer Professional

Prepared by: Bharath Raavi | LinkedIn: linkedin.com/in/br-r

NOTE

This study guide is an independent reference document designed for candidates preparing for the Databricks Certified Data Engineer Professional examination

Official Databricks Exam Prep Guide: Check out here
Official Databricks AI Prep Guide: Check out here

Table of content

Select a section below to navigate directly to its content.

Section 1 — Developing Code for Data Processing using Python and SQL (23%)

1.1. Python project structure and Declarative Automation Bundles
1.2. Dependency troubleshooting across environments
1.3. Python, Pandas, SQL, and Unity Catalog UDFs
1.4. Lakeflow Declarative Pipelines and Auto Loader
1.5. Lakeflow Jobs: UI, API, and CLI
1.6. Streaming tables vs. materialized views
1.7. AUTO CDC and slowly changing dimensions
1.8. Spark Structured Streaming vs. Apache Spark Declarative Pipelines
1.9. If/Else and For Each control flow in Lakeflow Jobs
1.10. Compute, environments, performance mode, and retries
1.11. Unit tests, integration tests, DataFrame.transform, and schema assertions
1.12. Stateful Structured Streaming: watermarks, output modes, foreachBatch, and checkpoints

Section 2 — Data Ingestion & Acquisition (12%)

2.1. Ingest different data formats and sources
2.2. Configure incremental CDC pipelines with Delta or Iceberg targets
2.3. Ingest CDC from relational databases with Lakeflow Connect
2.4. Implement OpenSharing and Clean Rooms
2.5. Configure Lakehouse Federation with governance

Section 3: Data Manipulation (12%)

3.1. Advanced transformations with Spark SQL and PySpark
3.2. Model and query semi-structured data using VARIANT
3.3. Apply AI functions, including ai_query, for inference in data pipelines
3.4. Implement data quality expectations in Lakeflow Declarative Pipelines

Section 4: Monitoring and Alerting (10%)

4.1 System tables: cost, compute, access and Lakeflow
4.2 REST APIs, CLI and SDK for job and pipeline monitoring
4.3 Pipeline event logs
4.4 Lakehouse alerts
4.5 Monitor Lakeflow Jobs in the UI and API

Section 5 — Cost & Performance Optimization (15%)

5.1 Unity Catalog managed tables and Predictive Optimization
5.2 Choose deletion vectors, Liquid Clustering or CLUSTER BY AUTO
5.3 Caching for repeated reads
5.4 Change Data Feed (CDF) for incremental processing
5.5 Query Profile: find the actual bottleneck
5.6 Liquid Clustering vs. partitioning and ZORDER

Section 6 — Ensuring Data Security and Compliance (8%)

6.1 Least-privilege access control
6.2 Attribute-based access control (ABAC), governed tags, row filters and column masks
6.3 Anonymization and pseudonymization
6.4 Detect and mask PII in batch and streaming pipelines
6.5 Data purging and retention compliance

Section 7 — Data Governance (5%)

7.1 Tags and comments for data discoverability
7.2 Unity Catalog permission inheritance

Section 8 — Debugging and Deploying (10%)

8.1 Diagnose failures with Spark UI, logs, system tables and Query Profile
8.2 Repair failed jobs and override parameters
8.3 Debug Lakeflow pipelines and Spark workloads with event logs and Spark UI
8.4 Deploy resources with Declarative Automation Bundles
8.5 Git-based CI/CD and Databricks Git folders
Common mistakes

Section 9 — Data Modeling (5%)

9.1 Design scalable Delta/Iceberg table layouts
9.2 Dimensional modeling, Materialized Views and Metric Views

Section 1 — Developing Code for Data Processing using Python and SQL (23%)

1.1. Python project structure and Declarative Automation Bundles

Official docs: Bundles overview · Bundle configuration · Bundle lifecycle · Bundle resources · Bundle templates

Core concept

A Declarative Automation Bundle (formerly known as **Databricks Asset Bundles**, or **DABs**) is a source-controlled definition of a Databricks project. It can contain Python files and notebooks, resource definitions such as **Lakeflow Jobs** and pipelines, tests, and target-specific deployment configuration. The bundle is deployed as a unit.

How it works on Databricks

1. Create the bundle structure.
2. Define source files, resources, and deployment targets.
3. Validate the configuration.
4. Deploy to a workspace target.
5. Run and test the deployed resources.

Databricks documents the lifecycle and supports resource definitions in bundle configuration. A typical (illustrative, not mandatory) layout keeps business logic in an importable package so that it can be unit tested outside a notebook:

```
TEXT
my_project/
├── databricks.yml            # bundle name, variables, targets (dev / staging
/ prod)
├── resources/
│   ├── jobs.yml              # Lakeflow Jobs definitions
│   └── pipelines.yml         # Lakeflow Declarative Pipelines definitions
├── src/
│   ├── my_project/           # importable Python package (pure functions,
testable)
│   │   ├── __init__.py
│   │   └── transforms.py
│   └── notebooks/ingest.py   # thin entry points that call the package
├── tests/
│   ├── unit/test_transforms.py
│   └── integration/test_pipeline.py
└── pyproject.toml            # build metadata -> builds a wheel for deployment
```

Code example — databricks.yml with targets and variables

```yaml
bundle:
  name: sales_etl

include:
  - resources/*.yml            # pull in job / pipeline definitions

variables:
  catalog:
    description: Unity Catalog catalog the code writes to
    default: dev_catalog

targets:
  dev:
    mode: development          # prefixes resource names, pauses schedules, per-user paths
    default: true
    workspace:
      host: https://<dev-workspace>.cloud.databricks.com
  prod:
    mode: production           # stricter validation (e.g. requires explicit root_path)
    workspace:
      host: https://<prod-workspace>.cloud.databricks.com
      root_path: /Workspace/Shared/.bundle/${bundle.name}/${bundle.target}
    variables:
      catalog: prod_catalog    # same code, different environment value
    run_as:
      service_principal_name: <application-id>   # never run prod as a person
```

Code example — a job resource that uses the variable

```yaml
# resources/jobs.yml
resources:
  jobs:
    daily_sales_job:
      name: daily_sales_${bundle.target}
      schedule:
        quartz_cron_expression: "0 0 2 * * ?"   # 02:00 daily
        timezone_id: UTC
      tasks:
        - task_key: ingest
          notebook_task:
            notebook_path: ../src/notebooks/ingest.py
            base_parameters:
              catalog: ${var.catalog}          # injected per target
```

Code example — the CLI lifecycle

```
CLI / SHELL
databricks bundle init                    # scaffold from a template
databricks bundle validate -t dev         # schema + reference checks (always run first)
databricks bundle deploy   -t dev         # upload files, create/update resources
databricks bundle run daily_sales_job -t dev   # trigger a deployed job or pipeline
databricks bundle summary  -t dev         # list deployed resources and their URLs
databricks bundle destroy  -t dev         # remove everything the bundle deployed
```

When to use it vs. alternatives

- Use a bundle when you want repeatable deployment, code review, source control, and CI/CD for a multi-resource project.
- Use the workspace UI directly for quick exploration or a small manual task that does not need a source-controlled deployment workflow.
- Use a bundle template when you want a consistent starting structure for repeated projects.

Approach	Strength	Limitation
Workspace UI	Fastest for exploration	No version history, hard to reproduce in another environment
REST API / CLI scripts	Fine-grained programmatic control	You maintain the create/update/delete logic yourself
Declarative Automation Bundles	Declarative, reviewable, multi-environment, CI/CD friendly	Requires CLI setup and a project structure
Terraform provider	Infrastructure-level resources (workspaces, UC objects)	Heavier for application-level jobs and pipelines

Common mistakes

- Keeping business logic tightly coupled to a single notebook when it needs to be reused and tested.
- Treating the bundle as only Python source code, rather than a project definition that can include resources and deployment configuration.
- Deploying without validating the bundle configuration.
- Assuming the illustrative folder layout above is mandatory.

> **[EXAM TIP]** If the requirement mentions repeatable deployment of jobs or pipelines across development and production, think Declarative Automation Bundles. `mode: development` is for dev targets (prefixed names, paused schedules); `mode: production` enforces production guardrails.

> **[PRODUCTION BEST PRACTICE]** Parameterize everything environment-specific (catalog, schema, cluster policy, notification emails) with bundle `variables` and per-target overrides. Deploy production only from CI using a service principal, and never hand-edit bundle-managed resources in the UI - the next `bundle deploy` will overwrite the drift.

1.2. Dependency troubleshooting across environments

Official documentation: Configure the serverless environment · Declarative Automation Bundles

Core concept

A Python dependency is available only if it is installed or otherwise made available in the environment that actually executes the code. A package working on a developer's laptop does not prove that it is available to a **serverless** task, a pipeline, or a deployed job.

Databricks serverless notebook environments support dependencies specified in the Environment side pane. Dependency installation happens before notebook or task code executes.

How it works on Databricks

The installation method depends on the execution environment:

- Serverless notebook or job: configure supported dependencies in the serverless environment settings.
- Bundle-deployed project: ensure the dependency configuration is part of the deployed project and is supported by the target task environment.
- Pipeline: verify the pipeline's supported dependency configuration rather than assuming notebook-installed libraries are inherited.
- Wheel files: ensure a compiled wheel matches the target architecture. Databricks notes that native-extension wheels built for one architecture can fail on another.

When to use which package form

Dependency form	Consider it when
PyPI package	The package is available from a package index and its version can be specified
Local .whl file	You have a built Python distribution to install
Python project or source archive	You need to install a project from its source or project files, provided the target environment supports that form

The exact supported path and packaging depend on the execution environment. Check the serverless dependency documentation for the target you use.

Code example — declaring dependencies per environment

YAML

```yaml
# Serverless job: the environment is declared on the job and referenced per task
resources:
  jobs:
    etl_job:
      name: etl_${bundle.target}
      environments:
        - environment_key: default
          spec:
            environment_version: "3"          # check the currently supported version
            dependencies:
              - pandas==2.2.2                 # pin exact versions for reproducibility
              - /Volumes/main/libs/wheels/my_pkg-1.4.0-py3-none-any.whl
      tasks:
        - task_key: transform
          environment_key: default
          spark_python_task:
            python_file: ../src/notebooks/transform.py
```

PYSPARK / PYTHON

```python
# Interactive notebook or classic compute only. This runs at notebook scope.
%pip install my_pkg==1.4.0 --quiet
dbutils.library.restartPython()   # restart so the new version is importable

# Verify what the *executing* environment actually sees
import importlib.metadata as md
print(md.version("my_pkg"))
```

Common mistakes

- Installing a library in an interactive session and assuming a separate job has the same environment.
- Ignoring package-version conflicts or transitive dependencies.
- Using a native-extension wheel built for the wrong CPU architecture.
- Trying to repair dependency installation inside the notebook's Python code. Installation happens before that code runs.
- Assuming serverless supports classic-compute setup methods such as init scripts or compute policies for installing packages.

> **[EXAM TIP]** "Works locally, fails on serverless" → check the task's supported environment and dependency configuration first. `ModuleNotFoundError` on a job but not in your notebook almost always means the library was installed at notebook scope only.

> **[PRODUCTION BEST PRACTICE]** Pin direct dependencies (`pkg==x.y.z`), build your own code into a versioned wheel in CI, publish it to a **Unity Catalog** volume or index, and reference that exact version from the bundle. Rebuild wheels per target architecture when they contain native extensions.

1.3. Python, Pandas, SQL, and Unity Catalog UDFs

Official documentation: What are UDFs? · Python scalar UDFs · SQL and Python UDFs in Unity Catalog

Core concept

A user-defined function (UDF) adds custom logic to a query or DataFrame operation. The main exam distinction is the execution model and scope of the function.

Type	Key idea	Typical consideration
Built-in Spark SQL function	Uses existing Spark functionality	Prefer it when it expresses the required logic
Python scalar UDF	Runs custom Python logic for scalar inputs and outputs	Useful when built-in functions are insufficient
Pandas UDF	Uses Pandas/Arrow-based batch processing for supported function types	Useful for vectorized logic over batches
Unity Catalog function	A governed function registered in Unity Catalog	Useful when the function should be shared and governed beyond one Spark session

Databricks documents Pandas UDF forms including Series-to-Series and iterator-based variants. Unity Catalog functions differ in scope and supported features from session-scoped PySpark UDFs.

How it works on Databricks

A session-scoped UDF is registered for the current Spark session. A Unity Catalog function is registered as a catalog object and has permissions such as `EXECUTE`, alongside the required catalog and schema usage permissions.

Example of a session-scoped scalar UDF:

PYSPARK / PYTHON
```
from pyspark.sql.functions import udf
from pyspark.sql.types import LongType

@udf(LongType())
def square_value(value):
    if value is None:          # always handle NULLs explicitly
        return None
    return value * value

df = spark.createDataFrame([(2,), (3,), (None,)], ["value"])
df.withColumn("squared", square_value("value")).show()
```

This is a session-scoped example; it does not create a Unity Catalog function.

Code example — Pandas UDF (vectorized)

PYSPARK / PYTHON

```
import pandas as pd
from pyspark.sql.functions import pandas_udf

@pandas_udf("double")                        # Arrow batches: much faster than
row-at-a-time
def celsius_to_f(c: pd.Series) -> pd.Series:
    return c * 9.0 / 5.0 + 32.0              # vectorized over the whole batch

spark.range(5).selectExpr("cast(id as double) as c") \
     .withColumn("f", celsius_to_f("c")).show()
```

Code example — Unity Catalog SQL and Python functions

SQL

```
-- Governed SQL UDF: stored in Unity Catalog, usable from any supported compute
CREATE OR REPLACE FUNCTION main.utils.net_price(price DOUBLE, tax_rate DOUBLE)
RETURNS DOUBLE
RETURN price * (1 + tax_rate);

-- Governed Python UDF
CREATE OR REPLACE FUNCTION main.utils.mask_last4(s STRING)
RETURNS STRING
LANGUAGE PYTHON
AS $$
  if s is None:
      return None
  return "*" * max(len(s) - 4, 0) + s[-4:]
$$;

-- Permissions: EXECUTE on the function + USE CATALOG / USE SCHEMA on parents
GRANT EXECUTE ON FUNCTION main.utils.mask_last4 TO `analysts`;

SELECT main.utils.mask_last4(card_number) FROM main.sales.payments;
```

When to use vs. alternatives

- Use a built-in Spark function when possible; it avoids introducing custom Python execution for logic Spark already supports.
- Consider a Pandas UDF for supported vectorized operations on batches.
- Use a Unity Catalog function when the function needs governed, reusable access across supported workloads.
- Use a session-scoped UDF for logic scoped to a notebook or job session.

Common mistakes

- Forgetting to handle NULL values in Python UDF logic.
- Assuming a session-scoped UDF automatically exists in other sessions.
- Assuming a Unity Catalog function has identical compute support to every session-scoped UDF.
- Using custom Python code for an operation already available as a built-in Spark function.

> **[EXAM TIP]** "Reusable and governed across users/workloads" points toward Unity Catalog; "vectorized batch operation" may point toward a Pandas UDF. Python UDFs are opaque to the Catalyst optimizer, so built-ins always win when they can express the logic.

1.4. Lakeflow Declarative Pipelines and Auto Loader

Official documentation: Auto Loader and cloud-object-storage ingestion · Auto Loader production guidance · Lakeflow pipelines Python reference · Schema inference and evolution

Core concept

Auto Loader incrementally discovers and processes new files arriving in cloud storage through the Structured Streaming source named `cloudFiles`. **Lakeflow Declarative Pipelines** gives you a declarative way to define pipeline datasets and their dependencies while Databricks manages much of the pipeline execution and operational setup.

How it works on Databricks

A typical ingestion flow is:

1. New files arrive in cloud storage.
2. Auto Loader discovers files and incrementally processes them.
3. A pipeline defines how the incoming records are transformed and stored.
4. You can add schema evolution, data quality **expectations**, and monitoring.

Databricks recommends Auto Loader with Lakeflow pipelines for many production incremental-ingestion workloads.

Code example — Auto Loader with schema evolution and rescued data

PYSPARK / PYTHON

```
from pyspark.sql.functions import current_timestamp, col

source_path     = "/Volumes/main/raw/events_json"
schema_location = "/Volumes/main/ops/_schemas/events"       # stores the
inferred/evolved schema
checkpoint      = "/Volumes/main/ops/_checkpoints/events"   # one checkpoint
per query

raw = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", schema_location)
    .option("cloudFiles.inferColumnTypes", "true")            # infer real
types, not all-strings
    .option("cloudFiles.schemaHints", "amount DOUBLE, event_ts TIMESTAMP")  #
pin known types
    .option("cloudFiles.schemaEvolutionMode", "addNewColumns")# new columns ->
restart & evolve
    .load(source_path)
    .withColumn("_ingest_ts", current_timestamp())
    .withColumn("_source_file", col("_metadata.file_path"))   # lineage back to
the file
)

(raw.writeStream
    .option("checkpointLocation", checkpoint)
    .option("mergeSchema", "true")                            # let the Delta
target accept new columns
    .trigger(availableNow=True)                               # incremental
batch; stops when caught up
    .toTable("main.bronze.events"))
```

schemaEvolutionMode	Behavior
addNewColumns	Default when no schema is provided. The stream fails once on a new column, updates the schema in the schema location, and succeeds on restart.
rescue	Schema never evolves; unexpected or mistyped data lands in the `_rescued_data` column.
failOnNewColumns	The stream fails on new columns until you update the schema or hints manually.
none	Ignores new columns without evolving or rescuing them (use with an explicit schema).

Code example — the same ingestion as a declarative pipeline

PYSPARK / PYTHON

```
from pyspark import pipelines as dp

@dp.table(
    name="bronze_events",
    comment="Raw JSON events ingested incrementally with Auto Loader")
def bronze_events():
    return (spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .option("cloudFiles.inferColumnTypes", "true")   # pipeline manages the schema location
        .load("/Volumes/main/raw/events_json"))
```

SQL

```
-- SQL equivalent using read_files (Auto Loader under the hood)
CREATE OR REFRESH STREAMING TABLE bronze_events
AS SELECT *, _metadata.file_path AS _source_file
FROM STREAM read_files(
  '/Volumes/main/raw/events_json',
  format => 'json');
```

When to use vs. alternatives

Option	Best fit	Trade-off
Auto Loader	Incremental ingestion of large numbers of files; schema may evolve	Needs a **checkpoint** and schema location; some options to tune
COPY INTO	Simpler idempotent loads when file volume is relatively small, or when selectively reloading a subset of files matters	Does not scale as well to millions of files
Auto Loader + Structured Streaming	More direct control over streaming logic and sinks	You own more of the implementation
Auto Loader inside Lakeflow pipelines	Managed orchestration, expectations, monitoring	Less low-level control over the query

SQL

```
-- COPY INTO: idempotent (already-loaded files are skipped)
COPY INTO main.bronze.events
FROM '/Volumes/main/raw/events_json'
FILEFORMAT = JSON
COPY_OPTIONS ('mergeSchema' = 'true');
```

Common mistakes

- Treating schema inference and schema evolution as the same thing: inference helps establish a schema; evolution handles changes over time.
- Ignoring rescued data or unexpected schema changes.
- Misconfiguring file discovery or checkpoints, which can increase costs or cause reliability problems.
- Using a full refresh without checking whether the source still retains all historical records.

> **[EXAM TIP]** Large-scale incremental file ingestion with schema evolution → Auto Loader, often combined with Lakeflow Declarative Pipelines. Directory listing is the default discovery mode; file notification / file events mode scales better for very large or deeply nested directories.

> **[PRODUCTION BEST PRACTICE]** Always set `cloudFiles.schemaLocation` and a unique `checkpointLocation`, ingest into a bronze table with `_metadata.file_path` and an ingestion timestamp, and keep `_rescued_data` monitored by an expectation or alert. A full refresh of a streaming table re-reads the source and truncates the target - confirm the source still holds all history first.

1.5. Lakeflow Jobs: UI, API, and CLI

Official documentation: Configure and edit Lakeflow Jobs · Jobs REST API · Task dependencies / Run if · Declarative Automation Bundles

Core concept

Lakeflow Jobs orchestrates tasks into a workflow. A job can contain one or more tasks, dependencies between tasks, compute settings, parameters, schedules, and failure-handling behavior.

How it works on Databricks

You can create and manage jobs using:

- UI: convenient for interactive configuration and run monitoring.
- REST API or CLI: suitable for programmatic automation.
- Declarative Automation Bundles: suitable for managing job definitions as source-controlled project configuration.

Databricks documents UI, API, and CLI job management, and recommends using the newer Jobs API version 2.2 for new and existing clients rather than API 2.0.

A common workflow graph is:

```text
Ingest → Validate → Transform → Publish
```

Independent tasks can run in parallel where their dependencies permit it.

Code example — the same workflow three ways

YAML

```yaml
# 1) Bundle (recommended for production)
resources:
  jobs:
    medallion_job:
      name: medallion_${bundle.target}
      max_concurrent_runs: 1
      email_notifications:
        on_failure: ["data-oncall@example.com"]
      tasks:
        - task_key: ingest
          notebook_task: { notebook_path: ../src/notebooks/ingest.py }
        - task_key: validate
          depends_on: [{ task_key: ingest }]
          notebook_task: { notebook_path: ../src/notebooks/validate.py }
        - task_key: transform
          depends_on: [{ task_key: validate }]
          run_if: ALL_SUCCESS              # default; also
AT_LEAST_ONE_SUCCESS, ALL_DONE, ...
          notebook_task: { notebook_path: ../src/notebooks/transform.py }
```

CLI / SHELL

```bash
# 2) CLI
databricks jobs list --output json
databricks jobs run-now 123456789                      # trigger by job id
databricks jobs list-runs --job-id 123456789 --limit 5
databricks jobs get-run 987654321                      # task-level states for
one run

# 3) REST (Jobs API 2.2) - token comes from the environment, never hard-coded
curl -s -X POST "$DATABRICKS_HOST/api/2.2/jobs/run-now" \
  -H "Authorization: Bearer $DATABRICKS_TOKEN" \
  -d '{"job_id": 123456789, "job_parameters": {"run_date": "2026-10-24"}}'
```

PYSPARK / PYTHON

```python
# 4) Python SDK
from databricks.sdk import WorkspaceClient   # authenticates from env / profile
/ OAuth

w = WorkspaceClient()
run = w.jobs.run_now(job_id=123456789, job_parameters={"run_date": "2026-10-
24"}).result()
print(run.state.result_state)
```

When to use vs. alternatives

- Lakeflow Jobs: orchestrate multi-step ETL, scheduled processing, and task dependencies.
- Lakeflow Declarative Pipelines: define the pipeline datasets and their transformations.
- Bundles: define and deploy the resources that the workflow uses.

These are complementary rather than mutually exclusive: a job can orchestrate tasks, while a pipeline defines the data-processing logic.

Common mistakes

- Forgetting that every job needs a task and a compute resource, even if it is manually triggered rather than scheduled.
- Confusing a task dependency with a condition that decides whether a task should run.
- Hard-coding secrets into API requests or job configuration. Databricks warns against storing secrets in plain text.

> **[EXAM TIP]** UI for manual setup, API/CLI for programmatic management, and bundles for repeatable source-controlled deployment. A "pipeline task" inside a job triggers a Lakeflow pipeline update; the job orchestrates, the pipeline transforms.

> **[PRODUCTION BEST PRACTICE]** Authenticate automation with a service principal and OAuth (or workload identity federation in CI), store secrets in secret scopes, set `max_concurrent_runs`, notifications, and timeouts on every production job, and use job parameters rather than hard-coded dates.

1.6. Streaming tables vs. materialized views

Official documentation: Streaming tables · Materialized views · How pipelines refresh

Core concept

Both are managed pipeline datasets, but their refresh semantics differ.

Requirement	Better starting point	Why
Incrementally ingest new records	Streaming table	A normal refresh processes newly arrived records
Low-latency streaming transformations	Streaming table	Designed for incremental processing
Maintain the result of a query over source data	Materialized view	Refreshes to reflect the current result of its defining query
Aggregation or joins that must reflect upstream changes	Materialized view is often appropriate	It can recompute affected results, including when older source data changes

Streaming table vs. materialized view vs. regular view

Aspect	Streaming table	Materialized view	Regular view
Processing model	Append-oriented incremental; each row processed once	Query result kept correct; incremental or full recompute	Computed on every query
Source	Streaming source (append-only or CDC feed)	Any batch query over tables	Any query
Stores data	Yes (Delta)	Yes (precomputed)	No
Reacts to old-data changes	No (past rows not reprocessed)	Yes	Yes (always current)
Typical layer	Bronze / Silver ingestion	Gold aggregates, joins	Logical abstractions, security

How it works on Databricks

A normal streaming-table refresh processes new records using the current definition; it does not automatically recalculate previously appended rows. A materialized view refresh aims to keep the query result correct and may incrementally refresh or recompute the query, depending on the operation and compute environment.

For example, suppose an upstream customer record changes after daily sales totals have already been computed. If the output must reflect the updated customer dimension across the full result, a materialized view may be the better fit than an append-oriented streaming table.

Code example — SQL and Python

```sql
-- Streaming table: processes each new order exactly once as it arrives
CREATE OR REFRESH STREAMING TABLE silver_orders
AS SELECT order_id, customer_id, CAST(amount AS DECIMAL(12,2)) AS amount,
order_ts
FROM STREAM(bronze_orders)
WHERE order_id IS NOT NULL;

-- Materialized view: always reflects the CURRENT join, even if customers
change
CREATE OR REFRESH MATERIALIZED VIEW gold_revenue_by_region
AS SELECT c.region, DATE_TRUNC('DAY', o.order_ts) AS day, SUM(o.amount) AS
revenue
FROM silver_orders o JOIN silver_customers c USING (customer_id)
GROUP BY c.region, DATE_TRUNC('DAY', o.order_ts);
```

PYSPARK / PYTHON

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

@dp.table(name="silver_orders")                # streaming DataFrame -> streaming table
def silver_orders():
    return spark.readStream.table("bronze_orders").where("order_id IS NOT NULL")

@dp.materialized_view(name="gold_revenue_by_region")   # batch DataFrame -> materialized view
def gold_revenue_by_region():
    o, c = spark.read.table("silver_orders"), spark.read.table("silver_customers")
    return (o.join(c, "customer_id")
             .groupBy("region", F.date_trunc("day", "order_ts").alias("day"))
             .agg(F.sum("amount").alias("revenue")))
```

When to use vs. alternatives

- Choose streaming tables for incremental ingestion and transformations where processing new records is the main requirement.
- Choose materialized views when the output should reflect the current result of its defining query.
- Choose a regular view when you want a query definition rather than persisted precomputed results; a standard view computes its result when queried.

Common mistakes

- Assuming a changed streaming-table definition rewrites historical output automatically.
- Assuming a streaming table's joins always recalculate when dimension data changes. Databricks documents that streaming-table joins do not recompute when dimensions change.
- Assuming materialized views always fully recompute. On serverless pipelines, Databricks can use incremental refresh when supported and cost-effective; classic pipeline behavior differs.

[EXAM TIP] Low-latency incremental ingestion points toward a streaming table. "Always reflect the current query result, including upstream changes" points toward a materialized view. A stream-static join only looks up the dimension at processing time - later dimension changes do not rewrite old rows.

[PRODUCTION BEST PRACTICE] Use streaming tables for bronze/silver ingestion and materialized views for gold aggregates and joins. Run materialized views on serverless pipelines to benefit from incremental refresh, and check the `event log` (`planning_information`) to see which refresh technique was chosen.

1.7. AUTO CDC and slowly changing dimensions

Official documentation: AUTO CDC APIs · foreachBatch and Delta Lake

Core concept

Change data capture (CDC) represents inserts, updates, and deletes in source data. The Lakeflow **AUTO CDC** APIs simplify applying those changes to target tables and support slowly changing dimension (SCD) Type 1 and Type 2 patterns. Databricks recommends AUTO CDC instead of the older APPLY CHANGES API naming.

How it works on Databricks

The main decisions are:

- Keys: which columns identify the entity being changed.
- Sequence column: how to order changes correctly.
- Delete condition: how to recognize delete events, when applicable.
- SCD type: whether to preserve historical versions.

The API supports CDC feeds, and AUTO CDC FROM SNAPSHOT handles snapshot-based change detection.

SCD Type 1 vs. Type 2

Type	Result	Use it when
SCD Type 1	Keep the current state; prior values are overwritten	Historical versions are not needed in the target
SCD Type 2	Preserve historical versions with validity information	You need to query what a record looked like at an earlier time

The Python API uses `stored_as_scd_type="1"` or `stored_as_scd_type="2"`. For Type 2, the target includes validity columns such as `__START_AT` and `__END_AT`.

Code example — AUTO CDC in Python and SQL

PYSPARK / PYTHON

```python
from pyspark import pipelines as dp
from pyspark.sql.functions import col, expr

# 1) Declare the target streaming table (schema is derived from the flow)
dp.create_streaming_table("customers_silver")

# 2) Apply the CDC feed to it
dp.create_auto_cdc_flow(
    target="customers_silver",
    source="customers_cdc_bronze",                  # streaming source with change events
    keys=["customer_id"],                           # entity identity
    sequence_by=col("sequence_num"),                # ORDER of changes (not arrival time!)
    apply_as_deletes=expr("operation = 'DELETE'"),  # how deletes are signalled
    except_column_list=["operation", "sequence_num"],
    stored_as_scd_type=2,                           # keep history; adds __START_AT / __END_AT
)

# Snapshot-based sources (periodic full extracts) use the snapshot variant:
# dp.create_auto_cdc_from_snapshot_flow(target=..., source=..., keys=[...], stored_as_scd_type=2)
```

SQL

```sql
CREATE OR REFRESH STREAMING TABLE customers_silver;

CREATE FLOW customers_flow AS AUTO CDC INTO customers_silver
FROM STREAM(customers_cdc_bronze)
KEYS (customer_id)
APPLY AS DELETE WHEN operation = 'DELETE'
SEQUENCE BY sequence_num
COLUMNS * EXCEPT (operation, sequence_num)
STORED AS SCD TYPE 2;

-- Query the current rows of an SCD Type 2 target
SELECT * FROM customers_silver WHERE __END_AT IS NULL;
```

Code example — custom foreachBatch + MERGE (when you need full control)

PYSPARK / PYTHON

```
from delta.tables import DeltaTable
from pyspark.sql import Window, functions as F

def upsert_batch(batch_df, batch_id):
    # keep only the latest change per key inside this micro-batch
    w = Window.partitionBy("customer_id").orderBy(F.col("sequence_num").desc())
    latest = batch_df.withColumn("rn", F.row_number().over(w)).where("rn =
1").drop("rn")

    (DeltaTable.forName(spark, "main.silver.customers").alias("t")
        .merge(latest.alias("s"), "t.customer_id = s.customer_id")
        .whenMatchedDelete(condition="s.operation = 'DELETE'")
        .whenMatchedUpdateAll(condition="s.operation <> 'DELETE'")
        .whenNotMatchedInsertAll(condition="s.operation <> 'DELETE'")
        .execute())                        # MERGE is idempotent on retry for the
same batch

(spark.readStream.table("main.bronze.customers_cdc")
    .writeStream.foreachBatch(upsert_batch)
    .option("checkpointLocation",
"/Volumes/main/ops/_checkpoints/customers_merge")
    .start())
```

When to use vs. alternatives

- Use AUTO CDC when you have a CDC feed and need to apply changes to a Lakeflow pipeline target.
- Use AUTO CDC FROM SNAPSHOT when the source is a sequence of snapshots rather than a change-event feed.
- A custom streaming `foreachBatch` plus MERGE can handle specialized logic, but you take on more implementation details.

Common mistakes

- Using the wrong key columns.
- Using a sequence field that does not correctly order events.
- Choosing Type 1 when consumers need history, or Type 2 when only the current state is required.
- Using the legacy API name in new code without recognizing Databricks' recommendation to use AUTO CDC.

> **[EXAM TIP]** "Keep only the latest value" → Type 1. "Preserve history of changes" → Type 2. The `sequence_by` column must reflect source ordering (an LSN, version, or event timestamp) - ingestion time is a classic wrong answer because out-of-order events would overwrite newer data.

> **[PRODUCTION BEST PRACTICE]** Prefer AUTO CDC over hand-written MERGE: it handles out-of-order events, deletes, and SCD2 bookkeeping for you. Keep the raw CDC feed in a bronze table so that you can replay history if the target logic changes.

1.8. Spark Structured Streaming vs. Apache Spark Declarative Pipelines

Official documentation: Apache Spark Declarative Pipelines · Structured Streaming concepts · Production considerations for Structured Streaming · Lakeflow Python reference

Core concept

Spark Structured Streaming is a streaming processing engine using familiar Spark APIs. **Apache Spark Declarative Pipelines** (SDP) is a declarative framework for defining batch and streaming pipelines in SQL and Python; it manages dependencies and orchestration between pipeline components. Lakeflow pipelines are built on SDP and add Databricks-specific capabilities.

How they differ

Consideration	Structured Streaming	Declarative Pipelines / Lakeflow pipelines
Programming style	Explicit streaming DataFrame/API operations	Declare datasets and pipeline logic
Orchestration	You manage more of the job and query lifecycle	The framework organizes dataset dependencies and pipeline updates
Custom sink logic	Direct streaming sink APIs, including `foreachBatch`	Pipeline-managed flows and supported pipeline APIs
Databricks-specific features	Spark streaming APIs and Databricks integrations	Lakeflow adds features such as AUTO CDC and expectations
Checkpoints	You choose and manage checkpoint locations	Managed by the pipeline
Retries / recovery	You configure job retries and restarts	Pipeline handles retries and update lifecycle

Code example — same logic, two styles

PYSPARK / PYTHON

```
# Structured Streaming: you own the query, trigger, checkpoint, and sink
from pyspark.sql import functions as F

(spark.readStream.table("main.bronze.orders")
    .where("amount >= 0")
    .withColumn("processed_at", F.current_timestamp())
    .writeStream
    .option("checkpointLocation",
"/Volumes/main/ops/_checkpoints/silver_orders")
    .trigger(processingTime="1 minute")              # micro-batch every minute
    .toTable("main.silver.orders"))
```

PYSPARK / PYTHON

```
# Declarative pipeline: you declare the dataset; the framework runs it
from pyspark import pipelines as dp

@dp.table(name="silver_orders")
@dp.expect_or_drop("non_negative", "amount >= 0")    # expectation is Lakeflow-specific
def silver_orders():
    return spark.readStream.table("bronze_orders").withColumn("processed_at", F.current_timestamp())
```

When to use vs. alternatives

- Choose Structured Streaming when you need direct control of a streaming query or custom sink behavior.
- Choose Lakeflow Declarative Pipelines when you want a managed, declarative ETL pipeline with dependency orchestration and Lakeflow features.
- Choose Apache Spark Declarative Pipelines when portability across supported SDP runtimes is important. Databricks documents that Lakeflow pipelines are interoperable with SDP, but the two do not have identical feature sets.

Common mistakes

- Assuming all Lakeflow-only APIs are available in open-source Apache Spark Declarative Pipelines. The Python reference explicitly identifies APIs such as AUTO CDC as Lakeflow-only.
- Assuming declarative pipelines remove the need to understand incremental processing, data quality, or state.
- Treating the two frameworks as entirely unrelated; Lakeflow pipelines build on SDP.

> **[EXAM TIP]** Need explicit query/sink control → Structured Streaming. Need declarative pipeline management and Lakeflow-specific functionality (expectations, AUTO CDC, managed checkpoints) → Lakeflow pipelines.

1.9. If/Else and For Each control flow in Lakeflow Jobs

Official documentation: If/else condition task · For Each task · Task dependencies

Core concept

These are workflow-level control-flow tasks---not Python if statements or for loops inside a notebook.

- If/else condition: chooses which downstream tasks run based on a boolean condition.
- For Each: runs a nested task repeatedly, passing a different input for each iteration.

How they work on Databricks

If/Else example: An ingestion task reports `bad_records`. A condition checks whether `bad_records > 0`; downstream tasks can route to quarantine handling or normal processing. Job condition expressions can reference task values.

For Each example: A workflow processes a list of table names, passing one table name per iteration to the nested task. Independent iterations can run concurrently if they have no dependencies on each other.

Code example — bundle YAML with both patterns

PYSPARK / PYTHON

```python
# ingest.py - publish a task value that a downstream condition can read
bad = spark.table("main.bronze.orders_quarantine").count()
dbutils.jobs.taskValues.set(key="bad_records", value=bad)
```

YAML

```yaml
tasks:
  - task_key: ingest
    notebook_task: { notebook_path: ../src/notebooks/ingest.py }

  - task_key: check_bad_records                  # If/else condition task
    depends_on: [{ task_key: ingest }]
    condition_task:
      op: GREATER_THAN
      left: "{{tasks.ingest.values.bad_records}}"   # task value reference
      right: "0"

  - task_key: quarantine_review                  # runs only when the condition
is true
    depends_on: [{ task_key: check_bad_records, outcome: "true" }]
    notebook_task: { notebook_path: ../src/notebooks/quarantine.py }

  - task_key: publish                            # runs only when the condition
is false
    depends_on: [{ task_key: check_bad_records, outcome: "false" }]
    notebook_task: { notebook_path: ../src/notebooks/publish.py }

  - task_key: load_all_tables                    # For Each: one nested task per
input
    for_each_task:
      inputs: '["orders", "customers", "products"]'   # or a task-value
reference
      concurrency: 3                                  # max parallel iterations
      task:
        task_key: load_one_table
        notebook_task:
          notebook_path: ../src/notebooks/load_table.py
          base_parameters:
            table_name: "{{input}}"                   # current element
```

When to use vs. alternatives

- Use If/Else to branch a job based on a result or parameter.
- Use For Each to repeat a task for an array of inputs.
- Use ordinary Python control flow when the decision or loop belongs inside a single Python task rather than determining which workflow tasks execute.

Common mistakes

- Confusing Run if dependency conditions---which evaluate upstream task states---with the separate If/else task that evaluates a boolean expression.
- Supplying a For Each task with inputs in the wrong shape or assuming nested For Each tasks are supported. Databricks documents that a For Each task cannot itself be the nested task of another For Each.
- Forgetting that a condition task can use task values and parameters, not arbitrary Python variables from another task.

> **[EXAM TIP]** "Choose a path" → If/Else. "Run the same task for every item in a list" → For Each. "Run a cleanup task even if upstream failed" → that is `run_if` (for example `ALL_DONE`), not an If/else task.

1.10. Compute, environments, performance mode, and retries

Official documentation: Serverless jobs · Task configuration and retries · Serverless dependencies and high memory

Core concept

Compute configuration affects compatibility, startup time, memory, performance, cost, and failure handling. Select it based on the task's actual needs rather than assuming one configuration is best for every workload.

How it works on Databricks

Serverless job compute lets Databricks manage infrastructure, scaling, and runtime optimizations. Supported tasks can use serverless compute, but workloads must meet its compatibility requirements.

For serverless jobs, distinguish these settings:

Setting	What it affects	When to consider it
Standard performance mode	Uses less compute to reduce cost; can have higher startup latency	Workloads that can tolerate some additional startup time
Performance optimized	Faster job startup and execution for time-sensitive work	Workloads where latency matters more
High memory for notebook tasks	Increases memory allocated to the notebook's REPL	Notebook code that runs out of memory
Retry policy	Controls whether and how failed tasks run again	Workloads that need retries or must avoid repeated execution

Important high-memory distinction: Databricks documents the high-memory serverless notebook option as increasing REPL memory from 16 GB to 32 GB. It does not increase the Spark session's memory. This feature is documented as Public Preview.

Code example — job-level and task-level settings

```yaml
resources:
  jobs:
    nightly_etl:
      name: nightly_etl_${bundle.target}
      performance_target: STANDARD          # or PERFORMANCE_OPTIMIZED for latency-critical jobs
      timeout_seconds: 7200                 # fail runaway runs
      tasks:
        - task_key: load
          notebook_task: { notebook_path: ../src/notebooks/load.py }
          max_retries: 2                    # re-run on failure ...
          min_retry_interval_millis: 60000  # ... after waiting 1 minute
          retry_on_timeout: false           # a timeout usually signals a real problem
        - task_key: charge_customers        # NON-idempotent side effect
          depends_on: [{ task_key: load }]
          notebook_task: { notebook_path: ../src/notebooks/charge.py }
          max_retries: 0                    # never repeat blindly
          disable_auto_optimization: true   # serverless: no extra automatic retries
```

When to use vs. alternatives

- Serverless: prefer it for supported job tasks when you want Databricks to manage compute infrastructure.
- Classic compute: consider it when you need a configuration or workload capability that is not supported by serverless.
- High memory: use it for notebook-process memory pressure; do not assume it fixes Spark executor or distributed-memory problems.
- Retries: use them for recoverable failures when rerunning the operation is safe.

Aspect	Serverless	Classic (job clusters)
Startup	Fast, managed	Minutes to provision
Configuration	Limited, environment-based	Full control (instance types, policies, init scripts, Spark confs)
Cost model	Pay for work done; performance mode tunes cost vs. speed	Pay for provisioned VMs; you size and tune
Library install	Environment spec	Cluster libraries, init scripts

Common mistakes

- Confusing notebook REPL memory with Spark session memory.
- Choosing performance-optimized mode for every task without considering its cost trade-off.
- Assuming disabling a retry policy and disabling serverless auto-optimization are necessarily the same setting.
- Disabling retries for a non-idempotent workload without checking the consequences of a failed partial run.

Databricks serverless job auto-optimization can include additional retries. Databricks recommends leaving it enabled for critical workloads that should succeed at least once, but documents disabling it for workloads that must execute at most once, such as non-idempotent jobs.

> **[EXAM TIP]** A retry question is often really an idempotency question: would repeating the task after failure be safe? At-least-once (retries on) vs. at-most-once (auto-optimization disabled) is the underlying trade-off.

1.11. Unit tests, integration tests, DataFrame.transform, and schema assertions

Official documentation: PySpark testing utilities · Declarative Automation Bundles

Core concept

- Unit tests check a small component, such as one transformation, in isolation.
- Integration tests check whether multiple components work together, such as reading input data, transforming it, and writing to the intended target.
- `assertDataFrameEqual` compares DataFrame contents according to the supported assertion options.
- `assertSchemaEqual` checks schema equality according to the supported options.
- `DataFrame.transform` applies a function to a DataFrame and returns the function's result, making it useful for composing transformations.

How it works on Databricks

The test strategy should separate three kinds of correctness:

1. Values: are the transformed records correct?
2. Schema: are the column names and types what downstream code expects?
3. Integration: does the pipeline or workflow operate correctly with its dependencies and target resources?

Bundles can include unit and integration tests alongside the source files and resource definitions, supporting testing as part of deployment.

Code example — pytest with Databricks Connect

PYSPARK / PYTHON

```python
# src/my_project/transforms.py - pure functions are easy to test
from pyspark.sql import DataFrame, functions as F

def drop_negative(df: DataFrame) -> DataFrame:
    return df.where(F.col("amount") >= 0)

def add_tax(df: DataFrame, rate: float) -> DataFrame:
    return df.withColumn("total", F.round(F.col("amount") * (1 + rate), 2))
```

PYSPARK / PYTHON

```python
# tests/unit/test_transforms.py
import pytest
from databricks.connect import DatabricksSession
from pyspark.testing import assertDataFrameEqual, assertSchemaEqual
from pyspark.sql.types import StructType, StructField, LongType, DoubleType
from my_project.transforms import drop_negative, add_tax

@pytest.fixture(scope="session")
def spark():
    return DatabricksSession.builder.getOrCreate()    # or a local SparkSession
in CI

def test_pipeline_values(spark):
    src = spark.createDataFrame([(1, 100.0), (2, -5.0)], ["id", "amount"])
    # DataFrame.transform composes small functions into a readable chain
    out = src.transform(drop_negative).transform(add_tax, 0.10)
    expected = spark.createDataFrame([(1, 100.0, 110.0)], ["id", "amount",
"total"])
    assertDataFrameEqual(out, expected)               # ignores row order by
default

def test_schema_contract(spark):
    out = (spark.createDataFrame([(1, 1.0)], ["id", "amount"])
                .transform(add_tax, 0.0))
    expected = StructType([StructField("id", LongType()),
                           StructField("amount", DoubleType()),
                           StructField("total", DoubleType())])
    assertSchemaEqual(out.schema, expected)           # catches renamed /
retyped columns
```

CLI / SHELL

```
pytest tests/unit -q                  # fast, local or Databricks Connect
databricks bundle deploy -t staging   # integration tests need deployed
resources
databricks bundle run integration_test_job -t staging
```

When to use vs. alternatives

- Use a unit test for transformation logic with small, controlled inputs.
- Use schema assertions when downstream consumers depend on particular field names and types.
- Use integration tests for interactions between components and actual data resources.
- Use `DataFrame.transform` to compose reusable DataFrame transformations; it is not itself a testing framework.

Common mistakes

- Testing only row values and overlooking schema changes.
- Using production-sized data for a simple unit test.
- Assuming row order is guaranteed unless the test explicitly accounts for ordering.
- Treating a passing unit test as proof that permissions, job configuration, or pipeline deployment are correct.

> **[EXAM TIP]** `assertDataFrameEqual` tests the result; `assertSchemaEqual` tests the schema; `transform` helps compose the code being tested. Unit tests do not prove permissions or deployment are right - that is the job of integration tests.

1.12. Stateful Structured Streaming: watermarks, output modes, foreachBatch, and checkpoints

Official documentation: Structured Streaming concepts · Watermarks · Use foreachBatch · Checkpoints · Production considerations

Core concept

A stateless streaming query processes records without retaining intermediate state. A stateful query retains state for operations such as aggregations, stream-stream joins, and deduplication. That state must be managed so that it does not grow without bound and can be recovered after failure.

A. Watermarks

A watermark defines a threshold for how long the engine waits for late-arriving data in supported stateful operations. In Python, use `withWatermark()` on the event-time column.

For example, a watermark of 10 minutes communicates a lateness tolerance; records arriving later than that threshold may be dropped from the relevant stateful operation.

Trade-off:

- Shorter watermark → less state and potentially lower latency, but less tolerance for late data.
- Longer watermark → more tolerance for late data, but more retained state and potentially higher latency and memory requirements.

```
PYSPARK / PYTHON
from pyspark.sql import functions as F

events = (spark.readStream.table("main.bronze.sensor_events")
    .withWatermark("event_time", "10 minutes"))      # tolerate up to 10 min of
lateness

# Windowed aggregation: state is dropped once the watermark passes the window
end
agg = (events
    .groupBy(F.window("event_time", "5 minutes"), "device_id")
    .agg(F.avg("temperature").alias("avg_temp"), F.count("*").alias("n")))

(agg.writeStream
    .outputMode("append")                            # emit each window once it
is final
    .option("checkpointLocation", "/Volumes/main/ops/_checkpoints/sensor_agg")
    .toTable("main.silver.sensor_5min"))

# Streaming de-duplication that also expires state via the watermark
deduped = events.dropDuplicatesWithinWatermark(["event_id"])

# Stream-stream join: BOTH sides need watermarks and a time-range condition
clicks =
(spark.readStream.table("main.bronze.clicks").withWatermark("click_time", "20
minutes"))
impr   =
(spark.readStream.table("main.bronze.impressions").withWatermark("impr_time",
"10 minutes"))
joined = impr.alias("i").join(clicks.alias("c"), F.expr("""
    i.ad_id = c.ad_id AND
    c.click_time BETWEEN i.impr_time AND i.impr_time + INTERVAL 15 MINUTES"""),
"leftOuter")
```

B. Output modes

Output mode controls how query results are written to the sink. The supported modes depend on the query and sink.

Mode	General meaning
append	Write newly appended result rows
update	Write rows whose results have changed
complete	Write the complete result table on each trigger

For windowed aggregations with watermarks, append mode can delay output until the watermark threshold passes; update mode can emit changed results sooner; complete mode retains the aggregation state and rewrites the result table on each trigger.

> **[EXAM TIP]** Exam trap: complete mode is not automatically the best choice for stateful processing. It can retain all aggregation state and rewrite the complete result each trigger. Append mode on an aggregation requires a watermark.

C. foreachBatch

`foreachBatch` lets you apply a function to each micro-batch, which is useful for operations such as custom writes or **Delta Lake** MERGE operations.

However, `foreachBatch` provides at least once write behavior by default rather than an unconditional exactly-once guarantee. If the same batch can be retried, design the sink operation to be idempotent or use a documented deduplication mechanism.

PYSPARK / PYTHON

```
def write_batch(batch_df, batch_id):
    # Idempotent Delta append: Delta skips a (txnAppId, txnVersion) pair it
already committed
    (batch_df.write.format("delta")
        .option("txnAppId", "orders_silver_writer")   # stable identifier of
this writer
        .option("txnVersion", batch_id)               # monotonically
increasing per batch
        .mode("append")
        .saveAsTable("main.silver.orders"))

(spark.readStream.table("main.bronze.orders")
    .writeStream.foreachBatch(write_batch)
    .option("checkpointLocation",
"/Volumes/main/ops/_checkpoints/orders_silver")
    .start())
```

Common mistakes:

- Performing a non-idempotent MERGE or write that produces duplicate effects on retries.
- Using `foreachBatch` to write to multiple sinks without accounting for serialization and added latency.
- Failing to consume the entire batch DataFrame when using stateful operators that require it.

D. Checkpoints and recovery

A checkpoint stores information needed to recover a streaming query, including source offsets, committed micro-batches, state for stateful operations, and query metadata. Each query should have its own checkpoint location. Changing to a new location causes the query to begin fresh because the previous checkpoint state is not there.

Important distinction: checkpointing is central to fault recovery, but exactly once behavior also depends on the source and sink semantics. A checkpoint alone does not make arbitrary external side effects exactly once. For example, `foreachBatch` writes need appropriate idempotency handling.

Change to a running query	Safe with the same checkpoint?	Notes
Change a filter or projection	Usually yes	Stateless logic; recovers from offsets
Change the aggregation keys, window, or join keys	No	State schema no longer matches; start a new checkpoint and reprocess
Change the output sink or table	Depends	Prefer a new checkpoint with a controlled backfill
Change watermark delay	Allowed, but semantics change	Results may differ from earlier batches
Delete or reuse a checkpoint across queries	No	Causes reprocessing or corrupt recovery

When to use vs. alternatives

- Watermarks: stateful event-time processing with bounded lateness.
- Output mode: choose according to whether the sink needs new rows, changed rows, or the full result.
- `foreachBatch`: custom micro-batch processing or writes not directly expressed through a standard streaming sink.
- Checkpoints: recovery and progress tracking for streaming queries.

If you want to avoid owning as much of the production streaming lifecycle and orchestration, consider whether Lakeflow Declarative Pipelines better fits the workload.

> **[EXAM TIP]** For a stateful query that must tolerate late data and recover after failure, look for a correctly chosen watermark, a suitable output mode, a unique durable checkpoint, and an idempotent sink strategy where required.

> **[PRODUCTION BEST PRACTICE]** Store checkpoints in a Unity Catalog volume or durable cloud storage that never gets cleaned up with the data, one per query. For stateful workloads consider RocksDB state store (`spark.sql.streaming.stateStore.providerClass`) to keep large state out of the JVM heap, and monitor state size in the streaming query progress.

Section 2 — Data Ingestion & Acquisition (12%)

This section tests whether you can pick the right ingestion path (files, message buses, CDC, sharing, federation) and know the operational prerequisites for each.

2.1. Ingest different data formats and sources

Exam objective: Develop ingestion pipelines for Delta Lake, Parquet, JSON, Iceberg, CSV, and Binary from message buses such as Kafka, Kinesis, and Pub/Sub, as well as cloud storage.

Official references: Auto Loader · Auto Loader options · Structured Streaming · Kafka integration · Databricks documentation

Core concept

Ingestion is the process of getting source data into your lakehouse in a form that downstream pipelines can process reliably. The right approach depends on both the data format and where the data comes from.

- File-based ingestion: Data arrives in cloud storage as files. Auto Loader is a common choice for incrementally discovering and ingesting new files.
- Message-bus ingestion: Events arrive continuously from a system such as Kafka; a streaming source reads events as they arrive.
- Table-based ingestion: Data already exists in a table format, such as Delta Lake or Iceberg, so compatibility and the source table's capabilities matter.

How it works on Databricks

For files, Auto Loader uses the `cloudFiles` source to discover new files and incrementally process them. You typically write the result into a Delta table and configure a schema location and checkpoint location for production streaming ingestion.

For a message bus, Spark Structured Streaming uses a source connector---for example, Kafka---and processes events as a stream. Cloud credentials, network access, and connector-specific options must be configured for your environment.

When to use which approach

Situation	Starting point
New files continually arrive in cloud storage	Auto Loader
Events continuously arrive from Kafka	Structured Streaming with the Kafka source
A one-time or scheduled file load is sufficient	Batch read with Spark
Data is already in Delta or Iceberg tables	Use the supported table-reading or ingestion path for that source format

Format cheat sheet

Format	How to ingest	Watch out for
Delta	`spark.readStream.table(...)` / `COPY INTO`	Streaming reads fail on upstream updates/deletes unless `skipChangeCommits` or CDF is used
Parquet	Auto Loader `cloudFiles.format = parquet`	Schema is embedded; evolution still needs a mode
JSON	Auto Loader `json` (or `VARIANT` for varying shapes)	Inconsistent types; use schema hints and `_rescued_data`
CSV	Auto Loader `csv` with `header`, `delimiter`	Header handling, quoted newlines (`multiLine`), all-string inference unless `inferColumnTypes`
Iceberg	Read Iceberg tables via Unity Catalog (managed or foreign)	Feature parity with Delta differs; verify per runtime
Binary	`format("binaryFile")` or Auto Loader `binaryFile`	Large files; columns are path, modificationTime, length, content

Code example: ingest JSON files incrementally

PYSPARK / PYTHON

```
source_path     = "/Volumes/main/raw/events_json"
schema_path     = "/Volumes/main/ops/checkpoints/events_schema"
checkpoint_path = "/Volumes/main/ops/checkpoints/events_stream"

raw_events = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("cloudFiles.schemaLocation", schema_path)   # required for schema
inference/evolution
    .load(source_path))

(raw_events.writeStream
    .format("delta")
    .option("checkpointLocation", checkpoint_path)      # progress + exactly-
once file tracking
    .trigger(availableNow=True)                         # drain what is
available, then stop
    .toTable("main.bronze.events"))
```

`availableNow=True` processes the data currently available and stops; it is useful for incremental scheduled runs, not a continuously running stream. The catalog, schema, volume paths, permissions, and source data must exist before running this example.

Code example: other file formats and Delta/Iceberg sources

PYSPARK / PYTHON

```python
# CSV with Auto Loader
csv_stream = (spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "csv")
    .option("header", "true")
    .option("cloudFiles.inferColumnTypes", "true")
    .option("cloudFiles.schemaLocation", "/Volumes/main/ops/_schemas/csv_in")
    .load("/Volumes/main/raw/csv_in"))

# Binary files (images, PDFs): metadata + raw bytes
binary_df = (spark.read.format("binaryFile")
    .option("pathGlobFilter", "*.pdf")
    .load("/Volumes/main/raw/documents"))          # path, modificationTime,
length, content

# Delta source as a stream; skip upstream UPDATE/DELETE commits instead of
failing
delta_stream = (spark.readStream
    .option("skipChangeCommits", "true")
    .table("main.silver.customers"))

# Iceberg table registered in Unity Catalog is read like any other table
iceberg_df = spark.read.table("main.lake.events_iceberg")
```

Code example: message buses

PYSPARK / PYTHON

```
from pyspark.sql import functions as F

# ---- Kafka ----
kafka_raw = (spark.readStream.format("kafka")
    .option("kafka.bootstrap.servers", dbutils.secrets.get("kafka", "brokers"))
    .option("subscribe", "orders")                       # or subscribePattern
/ assign
    .option("startingOffsets", "earliest")               # only applies to a
NEW checkpoint
    .option("kafka.security.protocol", "SASL_SSL")
    .option("kafka.sasl.mechanism", "PLAIN")
    .option("kafka.sasl.jaas.config", dbutils.secrets.get("kafka", "jaas"))
    .option("maxOffsetsPerTrigger", 100000)              # rate-limit each
micro-batch
    .load())

orders = (kafka_raw
    .select(F.col("key").cast("string").alias("key"),
            F.from_json(F.col("value").cast("string"),
                        "order_id STRING, customer_id BIGINT, amount
DOUBLE").alias("v"),
            "topic", "partition", "offset", "timestamp")   # keep Kafka
metadata for lineage
    .select("key", "v.*", "topic", "partition", "offset", "timestamp"))

# ---- Kinesis ----
kinesis = (spark.readStream.format("kinesis")
    .option("streamName", "clickstream")
    .option("region", "us-east-1")
    .option("initialPosition", "trim_horizon")
    .load())                                             # data column is
binary: cast + parse

# ---- Pub/Sub (Google Cloud) ----
pubsub = (spark.readStream.format("pubsub")
    .option("subscriptionId", "orders-sub")
    .option("topicId", "orders")
    .option("projectId", "my-gcp-project")
    # service-account credentials are supplied through connector options /
secrets
    .load())
```

Common mistakes

- Using a batch read when the requirement is continuous incremental ingestion.
- Reusing a checkpoint location for a different streaming query or changing its meaning without planning recovery.

- Assuming that knowing the file format is enough: cloud access, schema handling, and the source connector also matter.
- Treating every listed format or message bus as interchangeable; source-specific support and configuration must be checked in the relevant documentation.

> **[EXAM TIP]** Files in cloud storage → Auto Loader; event streams → source-specific Structured Streaming connector. `startingOffsets` only matters for a brand-new checkpoint; after that the checkpoint decides where to resume. Kafka, Kinesis and Pub/Sub give at-least-once delivery to the pipeline - make the sink idempotent.

> **[PRODUCTION BEST PRACTICE]** Land raw data untouched in bronze (keep Kafka key, topic, partition, offset, timestamp or the Auto Loader file path), parse in silver, and store broker credentials in secret scopes. Use `maxOffsetsPerTrigger` / `maxFilesPerTrigger` to protect the cluster during backfills.

2.2. Configure incremental CDC pipelines with Delta or Iceberg targets

Exam objective: Configure incremental change data capture (CDC) pipelines using Lakeflow Pipelines with Delta or Iceberg as the target table format.

Official references: Lakeflow CDC concepts / AUTO CDC · Lakeflow pipelines

Core concept

Change data capture (CDC) means applying changes from a source---such as inserted, updated, or deleted records---to a target without repeatedly rebuilding the entire target from scratch.

A CDC pipeline needs to determine which source changes are new, how to order them, and how to apply them to the correct target rows. Delta and Iceberg are table formats, but the exact supported pipeline features and configuration can differ by format.

How it works on Databricks

Lakeflow Declarative Pipelines provides declarative CDC functionality through APIs such as AUTO CDC. You define the source, keys, sequencing column, and how changes should be applied. For example, SCD Type 1 overwrites prior values, whereas SCD Type 2 preserves history by maintaining versions of records.

For this objective, don't assume every AUTO CDC feature is supported identically for both Delta and Iceberg. Check the current documentation for the precise target format and API combination before choosing the implementation.

Code example: declarative pipeline with AUTO CDC

PYSPARK / PYTHON

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col, expr

@dp.view(name="orders_cdc_clean")                          # deduplicate / type
the raw feed
def orders_cdc_clean():
    return (spark.readStream.table("main.bronze.orders_cdc")
            .where("order_id IS NOT NULL"))

dp.create_streaming_table("orders_current")                # SCD1 target: latest
state per key
dp.create_auto_cdc_flow(
    target="orders_current",
    source="orders_cdc_clean",
    keys=["order_id"],
    sequence_by=col("commit_ts"),                          # source commit time
/ LSN
    apply_as_deletes=expr("op = 'D'"),
    except_column_list=["op", "commit_ts"],
    stored_as_scd_type=1,
)
```

When to use CDC versus alternatives

- Use incremental CDC when you need to reflect ongoing inserts, updates, and deletes from an upstream system.
- Use append-only ingestion when records are immutable events and existing rows do not need to be updated or deleted.
- Use periodic full refresh or reloads when the source is small or change tracking is unavailable and a complete reload is acceptable.

Common mistakes

- Using a non-unique or incorrect key, which can cause changes to be applied to the wrong logical record.
- Using a sequencing field that cannot reliably establish change order.
- Choosing SCD Type 1 when historical versions are required, or SCD Type 2 when only the latest state is needed.
- Assuming CDC target-format support without verifying it in the official API documentation.

Code example: the core CDC idea

The following is a runnable PySpark illustration of applying a small batch of changes to a Delta table. It demonstrates the merge semantics behind CDC; it is not a complete Lakeflow AUTO CDC pipeline, and it does not by itself handle source offsets, event ordering, or SCD Type 2 history.

PYSPARK / PYTHON

```
from delta.tables import DeltaTable

# 1. Define the target table path/name
target_table = "main.silver.customers"

# 2. Create the initial target table with sample data
initial_data = [
    (1, "Asha"),
    (2, "Ben"),
]
(
    spark.createDataFrame(initial_data, ["customer_id", "name"])
    .write.format("delta")
    .mode("overwrite")
    .saveAsTable(target_table)
)

# 3. Define a small batch of incoming changes (updates + inserts)
incoming_changes = [
    (1, "Asha Rao"),   # Update existing customer
    (3, "Chen"),       # Insert new customer
]
updates_df = spark.createDataFrame(incoming_changes, ["customer_id", "name"])
updates_df.createOrReplaceTempView("customer_changes")

# 4. Perform the Delta Lake MERGE (upsert) operation
delta_target = DeltaTable.forName(spark, target_table)
(
    delta_target.alias("t")
    .merge(updates_df.alias("s"), "t.customer_id = s.customer_id")
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute()
)

# 5. Display the final merged state
display(spark.table(target_table).orderBy("customer_id"))
```

Expected final state: customer 1 is Asha Rao, customer 2 remains Ben, and customer 3 is inserted as Chen. Run this in a test schema where overwriting `main.silver.customers` is safe.

> **[EXAM TIP]** For the exam, know the declarative AUTO CDC approach as well as the distinction between it and a hand-written merge. A MERGE alone does not order out-of-sequence events or build SCD2 history; AUTO CDC does.

2.3. Ingest CDC from relational databases with Lakeflow Connect

Exam objective: Configure CDC ingestion pipelines from SQL Server, MySQL, and PostgreSQL using **Lakeflow Connect**.

Official references: Lakeflow Connect · SQL Server connector concepts · SQL Server source setup

Core concept

Lakeflow Connect provides managed ingestion connectors for supported sources. Instead of building and maintaining a custom extraction process, you configure the source connection and ingestion pipeline to bring source data into Databricks.

A key exam distinction is ingestion versus federation:

- Ingestion copies data into the lakehouse so it can be processed and stored there.
- **Lakehouse Federation** lets you query supported external data sources without first copying all of the data into Databricks.

How it works on Databricks

The general process is:

1. Prepare the source database and enable the required change-tracking or CDC mechanism.
2. Configure secure connectivity and credentials.
3. Create and configure the Lakeflow Connect ingestion pipeline.
4. Select the source objects and configure the destination.
5. Monitor ingestion, permissions, and failures.

The exact setup depends on the database. For example, Databricks documents SQL Server ingestion using TLS and JDBC, with credentials stored securely in Unity Catalog. The SQL Server connector can use Change Tracking or CDC, depending on source-table requirements.

Source	Typical source-side preparation	Notes
SQL Server	Enable Change Tracking or CDC on the database and tables; dedicated login with minimal rights	Primary key needed for change tracking; plan retention so changes are not purged before ingestion
MySQL	Enable binary logging with `binlog_format = ROW`; user with replication privileges	Binlog retention must exceed the longest expected pipeline outage
PostgreSQL	`wal_level = logical`, a publication, and a replication slot; user with replication role	Unconsumed replication slots hold WAL on the source - monitor disk

Code example: source preparation and pipeline definition

SOURCE DB SQL

```
-- SQL Server: enable Change Tracking (one option; CDC is the other)
ALTER DATABASE sales_db SET CHANGE_TRACKING = ON
  (CHANGE_RETENTION = 3 DAYS, AUTO_CLEANUP = ON);
ALTER TABLE dbo.customers ENABLE CHANGE_TRACKING;   -- table needs a primary
key
```

SQL

```
-- Databricks: Unity Catalog connection holds host + credentials (secrets, not
literals)
CREATE CONNECTION sqlserver_conn TYPE sqlserver
OPTIONS (
  host 'sql.example.internal',
  port '1433',
  user secret('lakeflow', 'sql_user'),
  password secret('lakeflow', 'sql_password')
);
```

YAML

```
# Managed ingestion pipeline defined as a bundle resource
resources:
  pipelines:
    sqlserver_ingest:
      name: sqlserver_ingest_${bundle.target}
      catalog: main
      schema: bronze
      ingestion_definition:
        connection_name: sqlserver_conn
        objects:
          - table:
              source_catalog: sales_db
              source_schema: dbo
              source_table: customers
              destination_catalog: main
              destination_schema: bronze
```

When to use Lakeflow Connect versus alternatives

- Lakeflow Connect: Prefer it when a supported database needs managed, incremental ingestion and the connector meets your requirements.
- JDBC batch ingestion: Consider it when periodic extracts are sufficient or the managed connector does not fit your source.
- Lakehouse Federation: Prefer querying the source in place when avoiding data movement is more important than maintaining a replicated target table.

Common mistakes

- Enabling a connector without first preparing the source database and its change-tracking features.
- Giving the ingestion identity overly broad database permissions.
- Confusing a federated query with a replicated, incrementally maintained target.
- Assuming the connector can access a private database without network and firewall configuration.
- Assuming all three database connectors have identical setup requirements.

Code example: inspect an ingested table

For managed connectors, creation and source configuration are generally workspace setup tasks, not a single portable notebook command. Once your connector pipeline has written a table to Unity Catalog, you can query it with:

PYSPARK / PYTHON

```
source_table = "main.bronze.source_customers"

df = spark.table(source_table)
print("Rows currently visible:", df.count())
df.printSchema()
display(df.limit(20))
```

Replace the table name with the actual destination configured for your connector. This example is runnable after the table exists and the current identity has read access; it inspects ingested data rather than configuring the connector itself.

> **[EXAM TIP]** Ingestion = copy + keep in sync (Lakeflow Connect). Federation = query in place. If the scenario says "incrementally replicate the operational database into the lakehouse with minimal custom code", the answer is Lakeflow Connect. Source preparation (CDC/binlog/logical replication) is the most common failure point.

2.4. Implement OpenSharing and Clean Rooms

Exam objective: Implement **OpenSharing** (Databricks-to-Databricks and Databricks-to-Open) and **Clean Rooms** for privacy-preserving collaboration.

Official references: OpenSharing recipient models · Create shares for OpenSharing · Databricks Clean Rooms overview

Core concept

These features help organizations share and analyze data across organizational boundaries without simply giving collaborators unrestricted access to internal systems. OpenSharing is built on the open **Delta Sharing** protocol.

- Databricks-to-Databricks (D2D): Share data with recipients who use Unity Catalog-enabled Databricks workspaces.

- Databricks-to-Open (D2O): Share data with recipients using other computing platforms through the OpenSharing protocol.
- Clean Rooms: Let multiple organizations collaborate on sensitive data in an isolated environment without directly exposing each other's underlying data.

How it works on Databricks

With OpenSharing, the provider creates a share, adds the intended data assets, creates or identifies recipients, and grants access. A share can include supported tables, views, volumes, notebooks, and other assets.

A Clean Room builds on OpenSharing to enable collaborative analysis in an isolated environment. Collaborators can contribute approved data assets and notebooks, while access to the underlying tables remains restricted. Clean Rooms require Unity Catalog, serverless compute, and OpenSharing to be enabled.

D2D vs. D2O vs. Clean Rooms

Aspect	Databricks-to-Databricks	Databricks-to-Open	Clean Rooms
Recipient	UC-enabled Databricks workspace	Any platform (Spark, pandas, Power BI, ...)	Multiple organizations, each with a UC workspace
Authentication	Recipient identified by metastore sharing identifier; no token	Token or credential file (bearer token / OIDC federation)	Collaborator identities inside the clean room
Data exposure	Recipient reads shared objects live (read-only)	Recipient reads shared objects live (read-only)	Raw data never exposed; only approved notebook outputs
Typical use	Partner or cross-region/cloud sharing	Sharing with non-Databricks consumers	Joint analytics on sensitive data (advertising, healthcare)

Code example: create a share, recipients, and grants

```sql
CREATE SHARE IF NOT EXISTS customer_analytics_share;

ALTER SHARE customer_analytics_share
  ADD TABLE main.gold.customer_summary
  WITH HISTORY;                              -- allow recipients to use time travel / streaming

-- Inspect the share and its contents.
DESCRIBE SHARE customer_analytics_share;
SHOW ALL IN SHARE customer_analytics_share;

-- D2D recipient: identified by the consumer metastore's sharing identifier
CREATE RECIPIENT IF NOT EXISTS partner_dbx
  USING ID 'aws:us-west-2:11111111-2222-3333-4444-555555555555';

-- D2O recipient: Databricks generates an activation link / credential file
CREATE RECIPIENT IF NOT EXISTS partner_open;
DESCRIBE RECIPIENT partner_open;           -- shows the activation URL for the open recipient

-- Creating a share does NOT give anyone access. This does:
GRANT SELECT ON SHARE customer_analytics_share TO RECIPIENT partner_dbx;
```

```sql
-- On the recipient (D2D) side: mount the share as a read-only catalog
CREATE CATALOG IF NOT EXISTS partner_data
USING SHARE provider_org.customer_analytics_share;

SELECT * FROM partner_data.gold.customer_summary LIMIT 10;
```

This creates the share and adds a table; it does not complete recipient setup or grant access to a recipient unless the recipient and `GRANT` steps are also completed.

When to use each

- D2D: The recipient is another Databricks organization and you want a governed, native sharing workflow.
- D2O: The recipient uses a different platform and needs to access shared data through an open protocol.
- Clean Rooms: Multiple parties need to collaborate on sensitive information, such as joint analytics, without granting each party direct access to the other's raw data.
- Data ingestion instead: Use ingestion when you need to create and maintain your own stored copy for independent downstream processing.

Common mistakes

- Assuming that creating a share automatically grants access to every recipient; recipient setup and permissions are separate steps.
- Treating OpenSharing as a Clean Room: sharing data and performing privacy-preserving collaborative analysis are related but different use cases.
- Assuming a Clean Room makes every analysis safe by default; the assets, notebooks, approvals, privileges, and collaboration design still matter.
- Forgetting that Clean Rooms require supported workspace features and serverless access.

> **[EXAM TIP]** Share = read-only live access without copying data to the recipient. Need to protect raw rows from the other party while still computing joint results → Clean Room. Recipient not on Databricks → D2O (token or credential file).

> **[PRODUCTION BEST PRACTICE]** Share governed gold tables or views (not raw tables), use `WITH HISTORY` only when consumers need streaming or time travel, set recipient token expiry, and review `system.access.audit` for share access events. Prefer sharing a view when you must hide columns or filter rows per recipient.

2.5. Configure Lakehouse Federation with governance

Exam objective: Configure Lakehouse Federation with governance across supported source systems, applying Unity Catalog permissions and connection-level credentials.

Official references: Lakehouse Federation overview · Foreign catalogs · Supported sources and connection setup

Core concept

Lakehouse Federation lets Databricks query supported external data systems without first copying all of their data into Databricks.

Unity Catalog provides the governance layer. A connection holds the information and credentials needed to connect to an external system; a **foreign catalog** represents an external database or catalog inside Unity Catalog, allowing users to discover and query its objects under governed access.

How it works on Databricks

The usual workflow is:

1. Create a Unity Catalog connection for the external source.
2. Configure its connection properties and credentials securely.
3. Create a foreign catalog using that connection.
4. Grant users the required Unity Catalog privileges.
5. Query the external tables through the foreign catalog.

Databricks documents support for sources including MySQL, PostgreSQL, SQL Server, Oracle, Snowflake, Amazon Redshift, and Google BigQuery. Support and requirements vary by source.

When to use Federation versus alternatives

Aspect	Lakehouse Federation	Lakeflow Connect	JDBC / Spark data source
Data location	Stays in the source system	Copied into Delta tables	Read into Spark on demand
Freshness	Live at query time	As fresh as the ingestion cadence	Live at read time
Governance	Unity Catalog grants on foreign catalog	Unity Catalog on the target tables	You handle credentials and access
Performance at scale	Limited by source and pushdown	Best (local Delta + Photon)	Depends on partitioned reads
Best for	Ad hoc, exploratory, low-volume queries; migration discovery	Production replication and heavy analytics	Unsupported sources, custom control

- Use Lakehouse Federation for live, read-only access to supported external data when minimizing data movement is important.
- Use Lakeflow Connect when you need managed ingestion and a maintained copy of data in the lakehouse. Databricks recommends Lakeflow Connect when both options support your source and performance at higher data volumes or lower latency is a priority.
- Use a Spark data source or JDBC when federation does not support your source, or when you need more direct control over reading and writing data.

Code example: create and query a foreign catalog

First, create a connection in Catalog Explorer or SQL and configure the required source-specific properties and credentials. Then, with the required privileges, run:

```sql
-- 1. Connection: holds endpoint + credentials (use secrets, never literals)
CREATE CONNECTION IF NOT EXISTS sales_db_connection TYPE postgresql
OPTIONS (
  host 'pg.example.internal',
  port '5432',
  user secret('federation', 'pg_user'),
  password secret('federation', 'pg_password')
);

-- 2. Foreign catalog: mirrors one external database inside Unity Catalog
CREATE FOREIGN CATALOG IF NOT EXISTS external_sales
USING CONNECTION sales_db_connection
OPTIONS (database 'sales');

-- Inspect available tables.
SHOW SCHEMAS IN external_sales;

-- 3. Governance: UC privileges control who can query the external data
GRANT USE CATALOG ON CATALOG external_sales TO `analysts`;
GRANT USE SCHEMA, SELECT ON SCHEMA external_sales.public TO `analysts`;

-- Example query; replace the schema and table names.
SELECT *
FROM external_sales.public.orders
LIMIT 10;
```

The query assumes the external source has a schema called `public` and a table called `orders`; adjust these names to match the source. The connection must already exist, the source must be reachable, and the current identity must have the necessary privileges.

Common mistakes

- Assuming a foreign catalog is a local copy of the source data; it represents external objects that are queried through the connection.
- Creating a connection but forgetting the privileges required to create or access a foreign catalog.
- Hardcoding credentials in notebooks instead of using the governed connection configuration.
- Forgetting that users need appropriate privileges on the Unity Catalog objects, not just network connectivity to the database.
- Using Federation when the requirement is actually to maintain an incrementally ingested target table.

> **[EXAM TIP]** "Query in place, no copy, governed by Unity Catalog" → Federation. "Keep a synchronized copy for heavy analytics" → Lakeflow Connect. Federation is read-only and query performance depends on pushdown to the source.

[PRODUCTION BEST PRACTICE] Give the connection to a platform team only (privilege on the connection), expose foreign catalogs to analysts with read-only grants, and monitor load on the source database - federated queries run against it live.

Objective summary

Objective	The key decision
1. Formats and sources	Files in cloud storage → often Auto Loader; event streams → source-specific Structured Streaming or pipeline connector
2. Incremental CDC	Select the correct CDC or snapshot approach, key, sequencing column, and SCD behavior
3. Relational ingestion	Lakeflow Connect moves and incrementally ingests data; source preparation and permissions matter
4. Sharing and Clean Rooms	OpenSharing shares assets; Clean Rooms support controlled collaboration across organizations
5. Federation	Query supported external sources in place through governed connections and foreign catalogs

Section 3: Data Manipulation (12%)

3.1. Advanced transformations with Spark SQL and PySpark

Official references: Spark SQL · PySpark DataFrames · Spark SQL window functions · Databricks query execution and performance

Core concept

Transformations turn raw records into useful datasets. Three techniques are especially important:

- Joins combine related rows from two datasets, such as orders and customers.
- Aggregations summarize rows, such as calculating revenue by customer.
- Window functions calculate values across related rows while keeping each row---for example, ranking orders within each customer or calculating a running total.

How it works on Databricks

You can express transformations in SQL or PySpark DataFrame APIs; both use Spark's distributed execution engine. Spark can optimize the execution plan, but joins and aggregations may require data shuffling between workers, so choosing appropriate join keys and avoiding unnecessary data movement matters.

When to use each technique

Technique	Use it when...	Example
Join	You need columns from related datasets	Add customer names to orders
Aggregation	You need a summary per group	Revenue per customer
Window function	You need a calculation across related rows without collapsing them	Rank each customer's orders

Code example — joins, aggregations, and windows

```sql
WITH order_enriched AS (
  SELECT o.order_id, o.customer_id, o.order_ts, o.amount, c.region
  FROM main.silver.orders o
  LEFT JOIN main.silver.customers c            -- keep orders even if the
customer is missing
    ON o.customer_id = c.customer_id
)
SELECT
  customer_id,
  order_id,
  amount,
  -- rank each customer's orders from largest to smallest (rows are NOT
collapsed)
  RANK()      OVER (PARTITION BY customer_id ORDER BY amount DESC)          AS
amount_rank,
  -- running total by time within customer
  SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_ts
                    ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)       AS
running_total,
  -- previous order amount for change analysis
  LAG(amount) OVER (PARTITION BY customer_id ORDER BY order_ts)              AS
prev_amount
FROM order_enriched
QUALIFY RANK() OVER (PARTITION BY customer_id ORDER BY amount DESC) <= 3     --
top 3 per customer
ORDER BY customer_id, amount_rank;
```

PYSPARK / PYTHON

```
from pyspark.sql import Window, functions as F
from pyspark.sql.functions import broadcast

orders    = spark.table("main.silver.orders")
customers = spark.table("main.silver.customers")          # small dimension

enriched = orders.join(broadcast(customers), "customer_id", "left")  #
broadcast avoids a shuffle

w_rank = Window.partitionBy("customer_id").orderBy(F.col("amount").desc())
w_run  = (Window.partitionBy("customer_id").orderBy("order_ts")
                .rowsBetween(Window.unboundedPreceding, Window.currentRow))

result = (enriched
    .withColumn("amount_rank",   F.rank().over(w_rank))
    .withColumn("running_total", F.sum("amount").over(w_run))
    .where("amount_rank <= 3")
    .orderBy("customer_id", "amount_rank"))               # explicit final sort

# Aggregation: revenue per customer and month
monthly = (orders.groupBy("customer_id", F.date_trunc("month",
"order_ts").alias("month"))
                 .agg(F.sum("amount").alias("revenue"),
F.countDistinct("order_id").alias("orders")))
```

Join strategy cheat sheet

Strategy	When Spark picks it / how to force it	Trade-off
Broadcast hash join	One side is small (under the autoBroadcast threshold); `broadcast(df)` or `/*+ BROADCAST(d) */`	No shuffle of the big side; small side must fit in executor memory
Shuffle hash join	Medium-sized build side	Shuffles both sides; hash table must fit in memory
Sort-merge join	Default for two large tables	Shuffles and sorts both sides; robust and scalable
Skew join (AQE)	Adaptive Query Execution splits skewed partitions automatically	Requires AQE enabled (default on Databricks)

Common mistakes

- Joining on the wrong key, creating duplicated or incorrect rows.
- Using an inner join when unmatched records should be preserved.
- Forgetting that grouping and aggregation reduce the number of rows.
- Defining a window without an appropriate `partitionBy` or ordering column.

- Assuming a window order automatically sorts the final output; use an explicit final sort when needed.

> **[EXAM TIP]** `RANK` leaves gaps on ties, `DENSE_RANK` does not, and `ROW_NUMBER` breaks ties arbitrarily - use `ROW_NUMBER` to deduplicate to exactly one row per key. A window without `PARTITION BY` pulls all rows into one partition (a performance pitfall).

> **[PRODUCTION BEST PRACTICE]** Filter and project as early as possible, join on typed keys of identical data types (implicit casts defeat optimizations), broadcast small dimensions, and inspect the plan with `EXPLAIN FORMATTED` or the **Query Profile** before reaching for cluster size.

3.2. Model and query semi-structured data using VARIANT

Develop a model and query semi-structured data using the **VARIANT** type and functions such as **parse_json** and **variant_get**.

Official references: VARIANT data type · parse_json · variant_get

Core concept

JSON data often has fields that vary from record to record. Databricks' `VARIANT` data type represents semi-structured values---including JSON objects, arrays, and scalars---without requiring you to define a fixed schema for every field in advance.

How it works on Databricks

- `parse_json(json_string)` parses a JSON string into a VARIANT value.
- `variant_get(variant_value, json_path, type)` extracts a value at a JSON path and casts it to the requested type.
- The `:` operator can also navigate into a VARIANT value using a JSON path.
- `try_parse_json` can return NULL for invalid JSON instead of raising a malformed-record parsing error.

For VARIANT and these functions, the cited Databricks documentation specifies Databricks SQL and Databricks Runtime 15.3 and above. Also note that Iceberg v2 tables do not support VARIANT columns; Iceberg v3 does.

When to use VARIANT versus alternatives

Representation	Best when	Trade-off
VARIANT	JSON records have varying shapes and you extract fields as you learn which matter	Less self-documenting; extraction needs paths and casts
STRUCT / typed columns	Structure is known and stable	Schema changes require evolution
STRING	You must preserve the raw payload or defer parsing	Repeated parsing at query time is slower
MAP	Homogeneous key-value pairs	All values must share one type

Code example — runnable in a Databricks SQL editor

```sql
WITH raw_events AS (
  SELECT *
  FROM VALUES
    ('{"customer_id": 101, "amount": 42.50, "region": "East"}'),
    ('{"customer_id": 102, "amount": 19.00}'),
    ('{"customer_id": 103, "amount": 75.25, "region": "West"}')
  AS t(payload)
),
parsed AS (
  SELECT PARSE_JSON(payload) AS event
  FROM raw_events
)
SELECT
  VARIANT_GET(event, '$.customer_id', 'BIGINT') AS customer_id,
  VARIANT_GET(event, '$.amount', 'DOUBLE')      AS amount,
  VARIANT_GET(event, '$.region', 'STRING')      AS region
FROM parsed;
```

The second record has no `region` field, so its extracted region is NULL. The example assumes the input strings are valid JSON. Note that `variant_get` takes a JSON path starting with `$.`.

Code example — operators, tolerant parsing, and Auto Loader

```sql
-- Colon operator + :: cast is the concise form of variant_get
SELECT event:customer_id::bigint AS customer_id,
       event:amount::double      AS amount,
       event:items[0].sku::string AS first_sku        -- array and nested
access
FROM parsed;

-- Tolerant parsing for dirty data: bad JSON becomes NULL instead of failing
the query
SELECT try_parse_json(payload) AS event FROM raw_events;

-- Flatten an array of objects into rows
SELECT o.order_id, i.value:sku::string AS sku, i.value:qty::int AS qty
FROM main.bronze.orders_variant o,
     LATERAL variant_explode(o.event:items) AS i;

-- Discover the shape of a variant column
SELECT schema_of_variant_agg(event) FROM parsed;
```

```
PYSPARK / PYTHON
# Ingest an entire JSON record into a single VARIANT column with Auto Loader
(spark.readStream.format("cloudFiles")
    .option("cloudFiles.format", "json")
    .option("singleVariantColumn", "raw")        # whole record -> VARIANT
column `raw`
    .load("/Volumes/main/raw/events")
    .writeStream.option("checkpointLocation",
"/Volumes/main/ops/_checkpoints/events_variant")
    .toTable("main.bronze.events_variant"))
```

Common mistakes

- Assuming a missing JSON path and an invalid cast are the same: a missing path returns NULL, while an incompatible cast can raise `INVALID_VARIANT_CAST`.
- Using `variant_get` with the wrong target type.
- Assuming malformed JSON always becomes NULL; `parse_json` raises an error for invalid JSON, while `try_parse_json` is designed for tolerant parsing.
- Forgetting that table-format compatibility matters when storing VARIANT columns.

[EXAM TIP] `parse_json` creates a VARIANT; `variant_get` extracts a path with a requested type. Remember missing-field behavior (NULL), casting errors (`INVALID_VARIANT_CAST`), and the Iceberg version limitation (v2 unsupported, v3 supported).

[PRODUCTION BEST PRACTICE] Keep the raw payload in a VARIANT bronze column and promote frequently filtered fields into typed columns in silver. Use `try_parse_json` when ingesting untrusted strings and route NULL results to a quarantine table.

3.3. Apply AI functions, including ai_query, for inference in data pipelines

Official objective: Apply AI functions such as **ai_query** to perform model inference in data pipelines for enrichment and classification.

Official reference: ai_query function

Core concept

Model inference means sending input data to an already available model and receiving a prediction or generated result. For data engineering, that can mean classifying customer feedback, summarizing text, extracting information, or enriching records with model-generated fields.

How it works on Databricks

`ai_query` invokes an existing Databricks **Model Serving** endpoint and parses its response. You can call it from SQL with an endpoint name and request; for custom models, you can also specify the expected return type.

The official documentation lists important prerequisites:

- The workspace must be in a supported Model Serving region.
- An appropriate model-serving endpoint must exist and have a model loaded.
- The invoking identity needs `CAN QUERY` permission on the endpoint.
- `ai_query` is unavailable on Databricks SQL Classic; the documentation specifies Databricks Runtime 15.4 LTS or above as required, and recommends 18.2 or above for the latest features and best performance.

When to use ai_query versus alternatives

- Use `ai_query` when a pipeline needs model-generated predictions or text-based enrichment as part of SQL processing.
- Use ordinary SQL rules or Python transformations when the task is deterministic and can be handled reliably without a model.
- Use a custom model endpoint when the inference task calls for a model you have deployed and `ai_query` supports its request and response schema.

The best choice depends on the task and available endpoint; the exam objective specifically calls out inference for enrichment and classification.

Code example — runnable once a supported endpoint is available

Run this in a supported Databricks SQL environment, replacing the endpoint name with one you can query.

```sql
SELECT
  feedback,
  ai_query(
    'your-model-serving-endpoint',
    CONCAT(
      'Classify this customer feedback as Positive, Negative, or Neutral. ',
      'Return only the label. Feedback: ',
      feedback
    )
  ) AS sentiment
FROM VALUES
  ('The delivery was fast and everything worked.'),
  ('The product arrived damaged and support did not help.'),
  ('The package arrived yesterday.')
AS feedback_data(feedback);
```

This example demonstrates the inference pattern. Its successful execution depends on the endpoint being available, accepting string requests, and returning a response compatible with the query. Model outputs are not guaranteed to follow the requested format, so production workflows should validate them.

Code example — production-oriented usage

SQL

```sql
-- Resilient batch enrichment: one bad row must not fail the whole query
SELECT
  review_id,
  ai_query(
    'your-model-serving-endpoint',
    CONCAT('Return exactly one word (Positive, Negative, Neutral): ',
review_text),
    failOnError => false,                                  -- returns a struct
with errorMessage
    modelParameters => named_struct('max_tokens', 10, 'temperature', 0.0)  --
deterministic
  ) AS result
FROM main.silver.reviews;

-- Persist results incrementally; do NOT re-call the model for rows already
scored
CREATE OR REFRESH MATERIALIZED VIEW main.gold.review_sentiment AS
SELECT review_id, review_text,
       ai_query('your-model-serving-endpoint',
                CONCAT('Classify sentiment as Positive, Negative or Neutral: ',
review_text)) AS sentiment
FROM main.silver.reviews;

-- Task-specific built-ins need no endpoint management
SELECT ai_analyze_sentiment(review_text) AS sentiment,
       ai_classify(review_text, ARRAY('billing', 'shipping', 'quality')) AS
topic
FROM main.silver.reviews;
```

PYSPARK / PYTHON

```python
# Same pattern from PySpark through expr(); validate model output before
trusting it
from pyspark.sql import functions as F

scored = (spark.table("main.silver.reviews")
    .withColumn("raw_label", F.expr(
        "ai_query('your-model-serving-endpoint', "
        "concat('Classify as Positive, Negative or Neutral: ', review_text))"))
    .withColumn("sentiment",
        F.when(F.col("raw_label").isin("Positive", "Negative", "Neutral"),
F.col("raw_label"))
         .otherwise(F.lit("Unknown"))))                  # guard against free-
form model output
```

Common mistakes

- Assuming `ai_query` trains a model; it calls an existing endpoint for inference.

- Using an endpoint name that does not exist or that the executing identity cannot query.
- Sending a request in the wrong format for the endpoint.
- Forgetting that model inference adds a service dependency and can fail; production pipelines need a deliberate error-handling and retry strategy.
- Assuming that the same endpoint and request syntax will work for every model type without checking its documented schema.

> **[EXAM TIP]** `ai_query` invokes a serving endpoint; it does not create or train the model. Be ready to identify endpoint prerequisites (`CAN QUERY`, supported region, supported compute), request/response schemas, and an appropriate inference use case.

> **[PRODUCTION BEST PRACTICE]** Control cost and latency: score only new or changed rows (streaming table or incremental materialized view), set low `temperature` for classification, use `failOnError => false` plus a retry/quarantine path, and log endpoint usage through the serving **system tables**.

3.4. Implement data quality expectations in Lakeflow Declarative Pipelines

Official objective: Implement data quality expectations to quarantine, drop, or fail on bad records.

Official references: Manage data quality with pipeline expectations · Python expectation decorators · Advanced expectation and quarantine patterns

Core concept

Data quality expectations are rules evaluated against records as they pass through a pipeline. For example, an order amount must be non-negative, or a customer ID must not be null.

Expectations let you decide what happens when a record violates a rule: keep it while recording quality metrics, drop it, or fail the update. Quarantining is a separate pattern for retaining invalid records in a separate dataset for investigation.

How it works on Databricks

The Python API uses decorators from `pyspark.pipelines`:

Behavior	Python API	What happens
Warn / keep	`@dp.expect(...)`	Invalid rows remain in the target, and quality metrics are recorded.
Drop	`@dp.expect_or_drop(...)`	Invalid rows are excluded from the target.
Fail	`@dp.expect_or_fail(...)`	The affected update fails when the rule is violated.

Expectations use Boolean SQL expressions. For example, `"amount >= 0"` is true for a non-negative amount and false for a negative amount.

For quarantine, Databricks documents a pattern that routes invalid rows to a separate dataset while keeping the main dataset clean.

When to use each behavior

- Warn / keep: You want visibility into data-quality problems without losing the records.
- Drop: Invalid rows must not reach downstream consumers, and retaining them in the main table is unnecessary.
- Fail: A violation means the update must not succeed---for example, a critical business invariant has been broken.
- Quarantine: You need to preserve bad records for diagnosis, correction, or reprocessing.

Code example — Lakeflow Declarative Pipelines Python

This example defines a streaming table that retains valid orders and drops records with missing IDs or negative amounts.

PYSPARK / PYTHON

```python
# dp.py
from pyspark import pipelines as dp

@dp.table(name="clean_orders")
@dp.expect_or_drop("valid_order_id", "order_id IS NOT NULL")
@dp.expect_or_drop("non_negative_amount", "amount >= 0")
def clean_orders():
    return spark.readStream.table("bronze_orders")
```

Before running: This is pipeline code, not a standalone notebook script. Create a Lakeflow Declarative Pipeline, ensure that `bronze_orders` is an available dataset input in the pipeline, and configure the source and target appropriately. The example drops violating records from `clean_orders`; it does not preserve them in a quarantine table.

Code example — grouped rules, SQL syntax, and a quarantine pattern

PYSPARK / PYTHON

```python
from pyspark import pipelines as dp
from pyspark.sql import functions as F

rules = {
    "valid_order_id":  "order_id IS NOT NULL",
    "non_negative":    "amount >= 0",
    "known_currency":  "currency IN ('USD', 'EUR', 'GBP')",
}
# A row is quarantined when it violates ANY rule (COALESCE so NULL counts as a failure)
is_bad = " OR ".join(f"NOT COALESCE({r}, FALSE)" for r in rules.values())

@dp.temporary_view()                                    # intermediate, not published
def orders_flagged():
    return (spark.readStream.table("bronze_orders")
            .withColumn("is_quarantined", F.expr(is_bad)))

@dp.table(name="orders_clean")
@dp.expect_all_or_fail({"not_flagged": "is_quarantined = false"})  # safety net
def orders_clean():
    return spark.readStream.table("orders_flagged").where("is_quarantined = false").drop("is_quarantined")

@dp.table(name="orders_quarantine", comment="Rejected rows kept for diagnosis and replay")
def orders_quarantine():
    return spark.readStream.table("orders_flagged").where("is_quarantined = true")
```

SQL

```sql
-- SQL syntax: constraints declare the action on violation
CREATE OR REFRESH STREAMING TABLE orders_clean (
  CONSTRAINT valid_order_id EXPECT (order_id IS NOT NULL) ON VIOLATION DROP ROW,
  CONSTRAINT non_negative   EXPECT (amount >= 0)          ON VIOLATION FAIL UPDATE,
  CONSTRAINT known_currency EXPECT (currency IN ('USD','EUR','GBP'))   -- warn only
)
AS SELECT * FROM STREAM(bronze_orders);
```

Common mistakes

- Assuming an expectation automatically quarantines a record; the default `expect` behavior keeps it.
- Dropping invalid records without preserving them when investigation or recovery is important.

- Using a rule that accidentally accepts NULL values when a field is mandatory.
- Expecting an expectation to orchestrate unrelated tasks; expectations enforce data quality, not general conditional workflow control.
- Assuming a failure always stops every flow in the pipeline: behavior differs between triggered and continuous pipelines.

[EXAM TIP] Know the difference between keeping, dropping, failing, and quarantining bad records---and remember that quarantine requires an explicit design. Expectation metrics (passed / failed record counts) are written to the pipeline `event log` under `flow_progress`.

[PRODUCTION BEST PRACTICE] Start new rules as warn-only (`expect`) to measure violation rates, then promote to drop or fail once you trust them. Always pair `expect_or_drop` on critical feeds with a quarantine table so dropped rows are recoverable.

Section 4: Monitoring and Alerting (10%)

Four tools, four questions. System tables answer "what happened across the account over time?" Job/pipeline APIs answer "what is the state of this run right now, programmatically?" The **event log** answers "what happened inside this pipeline update?" Alerts answer "tell me when a condition is met."

4.1 System tables: cost, compute, access and Lakeflow

Official docs: System tables reference · Billing system table · Jobs system tables

Core concept

System tables are Databricks-hosted, queryable records of account operations. They let you analyze historical billing, audit events, compute activity and job runs with SQL. They are not the same as live **Spark UI** metrics.

How it works

- `system.billing.usage` contains billable usage records.
- `system.access.audit` contains audit events.
- `system.lakeflow` contains job and pipeline monitoring data.
- System-table availability, retention and access requirements vary by table. Querying the tables still consumes compute.

Schema	Key tables	Typical question
system.billing	`usage`, `list_prices`	Which job or warehouse costs the most this month?
system.access	`audit`, `table_lineage`, `column_lineage`	Who accessed or changed this table? What depends on it?
system.compute	`clusters`, `node_timeline`, `warehouses`	Which clusters are under-utilized?
system.lakeflow	`jobs`, `job_run_timeline`, `job_task_run_timeline`, `pipelines`	Which jobs fail most often or run longest?
system.query	`history`	Which SQL queries are slow or expensive?

Code example: granting access and querying usage

```sql
-- System schemas are enabled by an account admin; access is then granted like any UC object
GRANT USE SCHEMA ON SCHEMA system.billing TO `finops`;
GRANT SELECT ON TABLE system.billing.usage TO `finops`;
GRANT SELECT ON TABLE system.billing.list_prices TO `finops`;
```

```
CLI / SHELL
# Enable a system schema (account/metastore admin)
databricks system-schemas enable <metastore-id> billing
databricks system-schemas list <metastore-id>
```

SQL example: recent DBU usage by day

```
SQL
SELECT
  usage_date,
  SUM(usage_quantity) AS total_dbus
FROM system.billing.usage
WHERE usage_unit = 'DBU'
  AND usage_date >= DATE_SUB(CURRENT_DATE(), 7)
GROUP BY usage_date
ORDER BY usage_date DESC;
```

This reports usage quantities, not dollars. Check the billing system table documentation for cost calculations and pricing joins.

Code example: estimated cost per job and audit trail

```
SQL
-- Estimated list-price cost per job over 30 days (usage quantity x list price)
SELECT
  u.usage_metadata.job_id                          AS job_id,
  SUM(u.usage_quantity * p.pricing.default)        AS est_cost_usd
FROM system.billing.usage u
JOIN system.billing.list_prices p
  ON  u.sku_name = p.sku_name
  AND u.cloud    = p.cloud
  AND u.usage_start_time >= p.price_start_time
  AND (u.usage_end_time <= p.price_end_time OR p.price_end_time IS NULL)
WHERE u.usage_date >= DATE_SUB(CURRENT_DATE(), 30)
  AND u.usage_metadata.job_id IS NOT NULL
GROUP BY ALL
ORDER BY est_cost_usd DESC
LIMIT 20;

-- Who dropped or altered a table in the last 7 days?
SELECT event_time, user_identity.email, service_name, action_name,
request_params
FROM system.access.audit
WHERE action_name IN ('deleteTable', 'updateTable')
  AND event_date >= DATE_SUB(CURRENT_DATE(), 7)
ORDER BY event_time DESC;

-- Impact analysis: what reads from this table?
SELECT DISTINCT target_table_full_name
FROM system.access.table_lineage
WHERE source_table_full_name = 'main.silver.orders';
```

When to use it vs. alternatives

Use system tables for account-wide historical analysis, dashboards and cost attribution. Use the Spark UI for detailed execution-stage diagnosis, and job-run pages for inspecting a particular run.

Common mistakes

- Assuming every system table has the same retention or availability.
- Forgetting to join usage records to relevant job/compute metadata when attributing costs.
- Treating usage quantity as a currency amount without applying the appropriate pricing data.
- Exporting sensitive audit data without restricting access.

> **[EXAM TIP]** Historical, account-wide, SQL-queryable → system tables. One failing run → job run page. Slow stage / skew / spill → Spark UI. DBUs are not dollars: join `list_prices` (and note list price differs from your negotiated rate).

> **[PRODUCTION BEST PRACTICE]** Tag jobs, warehouses and clusters with `custom_tags` (team, project, environment) so that `system.billing.usage.custom_tags` supports chargeback. Build a dashboard and a cost alert on top of the billing tables rather than reviewing invoices after the fact.

4.2 REST APIs, CLI and SDK for job and pipeline monitoring

Official docs: Monitor Lakeflow Jobs · Python SDK

Core concept

Databricks exposes jobs and other resources through APIs and command-line tooling so monitoring can be automated rather than performed only in the workspace UI.

How it works

Use the Jobs UI to inspect runs interactively, the CLI for scripts and automation, and the SDK/API to integrate run status into monitoring systems.

Code example: CLI, REST, and SDK

CLI / SHELL

```
databricks jobs list-runs --job-id 123456789 --limit 10 --output json
databricks jobs get-run 987654321 --output json | jq '.tasks[] | {task_key, state}'
databricks pipelines list-pipeline-events <pipeline-id> --max-results 20

# REST: recent runs with task detail
curl -s -H "Authorization: Bearer $DATABRICKS_TOKEN" \

"$DATABRICKS_HOST/api/2.2/jobs/runs/list?job_id=123456789&expand_tasks=true&limit=5"
```

PYSPARK / PYTHON

```
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.jobs import RunResultState

w = WorkspaceClient()                    # picks up OAuth / env credentials;
no hard-coded tokens

def failed_tasks(job_id: int, limit: int = 20):
    """Return (run_id, task_key, error) for failed tasks - not just failed
runs."""
    out = []
    for run in w.jobs.list_runs(job_id=job_id, limit=limit, expand_tasks=True):
        for t in (run.tasks or []):
            if t.state and t.state.result_state == RunResultState.FAILED:
                out.append((run.run_id, t.task_key, t.state.state_message))
    return out

for rid, task, msg in failed_tasks(123456789):
    print(f"run={rid} task={task}: {msg}")
```

When to use it vs. alternatives

Use an API or SDK for recurring checks, centralized reporting and CI/CD automation. Use the UI when you need to investigate a particular run's task graph, logs or output.

Common mistakes

- Checking only the overall job status and missing a failed task or a long-running task.
- Treating a one-time submitted run as though it were a saved job.
- Hardcoding credentials instead of using an approved authentication configuration.

> **[EXAM TIP]** A job can finish with a failed task that was configured not to fail the run (or a skipped branch). Always inspect task-level states. `runs/submit` creates a one-time run that does not appear as a saved job definition.

4.3 Pipeline event logs

Official docs: Monitor pipeline event logs · Pipeline event log schema

Core concept

A Lakeflow pipeline event log records pipeline activity, including progress, data-quality events, lineage and errors. The event log is a diagnostic source, not simply a list of final pipeline statuses.

How it works

Query the pipeline's event log and inspect timestamps, event types, levels and structured details. The `details` field can contain JSON whose structure depends on the event type.

Example query — replace the placeholder with your pipeline ID

```sql
SELECT
  timestamp,
  event_type,
  level,
  message,
  details
FROM event_log('YOUR-PIPELINE-ID')
ORDER BY timestamp DESC
LIMIT 100;
```

Use this in a context that supports the pipeline `event_log()` function and where you have access to that pipeline's event log.

Code example: data quality, latest update, and errors

```sql
-- Latest update id
CREATE OR REPLACE TEMP VIEW latest_update AS
SELECT origin.update_id
FROM event_log('YOUR-PIPELINE-ID')
WHERE event_type = 'create_update'
ORDER BY timestamp DESC LIMIT 1;

-- Expectation results (passed / failed records) for that update
SELECT
  row_expectations.dataset,
  row_expectations.name,
  SUM(row_expectations.passed_records) AS passing,
  SUM(row_expectations.failed_records) AS failing
FROM (
  SELECT explode(from_json(details:flow_progress.data_quality.expectations,

'array<struct<name:string,dataset:string,passed_records:int,failed_records:int>>'))
AS row_expectations
  FROM event_log('YOUR-PIPELINE-ID'), latest_update
  WHERE event_type = 'flow_progress' AND origin.update_id =
latest_update.update_id)
GROUP BY ALL;

-- Row counts written by each flow
SELECT origin.flow_name, details:flow_progress.metrics.num_output_rows AS rows_out
FROM event_log('YOUR-PIPELINE-ID')
WHERE event_type = 'flow_progress' AND details:flow_progress.status = 'COMPLETED';
```

When to use it vs. alternatives

Use event logs to understand what happened inside a pipeline. Use system tables to analyze broader job history and the Spark UI to investigate Spark execution behavior.

Common mistakes

- Assuming every event has the same details schema.
- Building long-term monitoring on fields the docs mark as evolving or deprecated.
- Reading only the latest event rather than following the sequence of events leading to an error.

> **[EXAM TIP]** Pipeline question + data quality metrics, lineage, or update errors → event log. The first error is usually the root cause; later errors are often consequences.

> **[PRODUCTION BEST PRACTICE]** Persist the event log of production pipelines to a governed table (publish it through the pipeline's event-log setting) and build dashboards and alerts on `flow_progress` metrics and expectation failures instead of querying ad hoc.

4.4 Lakehouse alerts

Official docs: Databricks SQL alerts · System tables reference

Core concept

The new exam guide explicitly calls out Databricks Lakehouse alerts for governed business metrics, data quality, anomaly signals, usage and cost, SQL warehouse/query health, audit/security events, AI agent quality and Lakeflow Jobs branching.

Do not automatically substitute the older, narrower SQL-alert objective for this new one. SQL alerts remain useful, but the exam guide names the broader Lakehouse alerts capability.

How it works

An alert evaluates a monitored condition and can notify people or systems when its criteria are met. For SQL alerts specifically, a query runs on a schedule and the alert condition is evaluated against the result.

SQL example — find orders with missing identifiers

```sql
SELECT COUNT(*) AS invalid_order_count
FROM main.analytics.orders
WHERE order_id IS NULL;
```

This query is the monitoring signal, not the alert configuration itself. In the relevant alert interface, configure the query, schedule and threshold to suit the requirement. The exact setup differs by alert type; consult the official alert documentation above.

Code example: alert signal queries for common categories

```sql
-- Cost: yesterday's DBUs far above the trailing 7-day average (trigger when spike_ratio > 1.5)
WITH daily AS (
  SELECT usage_date, SUM(usage_quantity) AS dbus
  FROM system.billing.usage WHERE usage_unit = 'DBU' AND usage_date >=
DATE_SUB(CURRENT_DATE(), 8)
  GROUP BY usage_date)
SELECT MAX(CASE WHEN usage_date = DATE_SUB(CURRENT_DATE(), 1) THEN dbus END)
       / AVG(CASE WHEN usage_date < DATE_SUB(CURRENT_DATE(), 1) THEN dbus END)
AS spike_ratio
FROM daily;

-- Data freshness: minutes since the last write to a gold table (trigger when > 120)
SELECT TIMESTAMPDIFF(MINUTE, MAX(order_ts), CURRENT_TIMESTAMP()) AS
minutes_stale
FROM main.gold.orders;

-- Jobs: failures in the last hour
SELECT COUNT(*) AS failed_runs
FROM system.lakeflow.job_run_timeline
WHERE result_state = 'FAILED' AND period_end_time >= CURRENT_TIMESTAMP() -
INTERVAL 1 HOUR;

-- Security: grants executed on sensitive schemas in the last day
SELECT COUNT(*) AS grant_events
FROM system.access.audit
WHERE action_name IN ('updatePermissions') AND event_time >=
CURRENT_TIMESTAMP() - INTERVAL 1 DAY;
```

When to use it vs. alternatives

Use alerts when you need a signal or notification. Use a dashboard for visual trend exploration and system tables for deeper historical analysis.

Common mistakes

- Creating a query that returns the wrong grain or more rows than the alert condition expects.
- Assuming an alert continuously evaluates data without a schedule or trigger.
- Confusing a business-metric alert with an alert for job execution failure.

> **[EXAM TIP]** An alert query should return a single, well-defined value (or one row per entity) that the condition compares to a threshold. For job-level failure notifications, use job notifications; for data or metric conditions, use alerts.

4.5 Monitor Lakeflow Jobs in the UI and API

Official docs: Monitor Lakeflow Jobs · Jobs system tables

Core concept

Job monitoring focuses on run status, task status, duration, error codes, run parameters and performance information.

How it works

The Lakeflow Jobs UI offers run history and task-level details. The Jobs API/SDK enables automated inspection. Job system tables support broader historical analysis and can be joined with billing information for cost investigations.

When to use it vs. alternatives

Use the UI for one run's execution path; use the API for automated checks; use system tables for multi-job trends and cost analysis.

SQL example: inspect recent job-run records

```sql
SELECT
  workspace_id,
  job_id,
  run_id,
  result_state,
  period_start_time,
  period_end_time
FROM system.lakeflow.job_run_timeline
WHERE period_start_time >= CURRENT_TIMESTAMP() - INTERVAL 7 DAYS
ORDER BY period_start_time DESC
LIMIT 100;
```

This assumes the relevant system table is available to your workspace and you have permission to query it. Use the documented schema for the exact columns in your environment.

Code example: failure rate, duration, and cost per job

```sql
-- Failure rate and average duration by job over 30 days
WITH runs AS (
  SELECT job_id, run_id,
         MAX(result_state)                                                AS
result_state,
         TIMESTAMPDIFF(SECOND, MIN(period_start_time), MAX(period_end_time)) AS
duration_s
  FROM system.lakeflow.job_run_timeline
  WHERE period_start_time >= CURRENT_TIMESTAMP() - INTERVAL 30 DAYS
  GROUP BY job_id, run_id)
SELECT job_id,
       COUNT(*)                                          AS runs,
       AVG(CASE WHEN result_state = 'FAILED' THEN 1 ELSE 0 END) AS
failure_rate,
       AVG(duration_s)                                   AS avg_duration_s
FROM runs
GROUP BY job_id
ORDER BY failure_rate DESC, avg_duration_s DESC;

-- Join to billing for cost per run (duration is not cost!)
SELECT usage_metadata.job_id, usage_metadata.job_run_id, SUM(usage_quantity) AS
dbus
FROM system.billing.usage
WHERE usage_metadata.job_id IS NOT NULL AND usage_date >=
DATE_SUB(CURRENT_DATE(), 30)
GROUP BY ALL;
```

Common mistakes

- Treating a job's overall outcome as a complete explanation of which task failed.
- Forgetting that run-history retention is limited.
- Confusing job duration with compute cost.

> **[EXAM TIP]** UI = one run; API/SDK = automation; system tables = trends and cost. Duration alone does not tell you cost - a short run on a huge cluster can cost more than a long run on a small one.

Section 5 — Cost & Performance Optimization (15%)

This section rewards diagnosing before optimizing: identify whether the bottleneck is file layout, **data skipping**, shuffle, skew, caching or compute sizing, and only then choose the feature.

5.1 Unity Catalog managed tables and Predictive Optimization

Official docs: Predictive Optimization for Unity Catalog managed tables · Managed tables

Core concept

Unity Catalog managed tables let Databricks manage more of the table's storage lifecycle and maintenance. **Predictive Optimization** can automatically run `OPTIMIZE`, `VACUUM` and `ANALYZE` when useful, reducing manual maintenance work.

How it works

Databricks evaluates managed tables for maintenance opportunities and can collect statistics and optimize file layout. Predictive Optimization does not mean every operation runs immediately on every table; operations are selected based on expected benefit.

When to use it vs. alternatives

Prefer managed tables when you want Unity Catalog to manage the table lifecycle and supported maintenance. External tables can be appropriate when you need to retain direct control over externally managed storage or share the storage lifecycle with other systems.

Aspect	Managed table	External table
Storage location	Managed by Unity Catalog in the metastore/catalog/schema storage root	Your own cloud path via an external location
Lifecycle on `DROP TABLE`	Data is removed (recoverable with `UNDROP` for a retention window)	Only metadata is removed; files remain
Predictive Optimization	Supported	Not supported
Governance and lineage	Full	Full, but you own storage layout and cleanup
Typical use	Default for new Delta/Iceberg tables	Data shared with non-Databricks readers or legacy paths

SQL example: create a managed Delta table from a query

```sql
CREATE OR REPLACE TABLE main.analytics.daily_sales
USING DELTA
AS
SELECT
  CAST(order_timestamp AS DATE) AS order_date,
  SUM(amount) AS total_sales
FROM main.raw.orders
GROUP BY CAST(order_timestamp AS DATE);
```

This example assumes `main.raw.orders` exists and you have privileges to read it and create the target table.

Code example: enabling and auditing Predictive Optimization

```sql
-- Enable at account default, catalog, or schema level (inherits downward)
ALTER CATALOG main ENABLE PREDICTIVE OPTIMIZATION;
ALTER SCHEMA main.analytics INHERIT PREDICTIVE OPTIMIZATION;

-- See what it actually did (OPTIMIZE / VACUUM / ANALYZE operations and their cost)
SELECT start_time, table_name, operation_type, usage_quantity
FROM system.storage.predictive_optimization_operations_history
ORDER BY start_time DESC LIMIT 50;

-- Compare with an external table definition
CREATE TABLE main.raw.partner_files
USING DELTA
LOCATION 's3://company-landing/partner_files';     -- external: you own file lifecycle

-- Recover an accidentally dropped managed table (within the retention window)
UNDROP TABLE main.analytics.daily_sales;
```

Common mistakes

- Assuming Predictive Optimization makes every table query optimal without regard to workload.
- Manually scheduling redundant maintenance without checking what automation already does.
- Treating a managed table and an external table as identical in lifecycle and operational responsibility.

> **[EXAM TIP]** Fewer manual `OPTIMIZE`/`VACUUM` jobs is the point of managed tables with Predictive Optimization. External table + "who deletes the files on DROP?" is a favorite trap: not Databricks.

> **[PRODUCTION BEST PRACTICE]** Default to managed tables, enable Predictive Optimization at the catalog level, and keep external tables for the specific cases that need external storage ownership. Review `predictive_optimization_operations_history` to verify it is saving more than it costs.

5.2 Choose deletion vectors, Liquid Clustering or CLUSTER BY AUTO

Official docs: Deletion vectors · Liquid Clustering and automatic clustering

Core concept

These features solve different problems:

- Deletion vectors: mark rows as deleted or updated without immediately rewriting the entire data file.
- **Liquid Clustering**: organizes table data to improve filtering and query performance, and can evolve as access patterns change.
- **CLUSTER BY AUTO**: lets Databricks automatically select clustering keys for supported managed tables based on workload patterns.

How it works

Deletion vectors reduce write amplification for certain row-level changes. Liquid Clustering optimizes data layout. Automatic clustering uses workload history and predictive optimization to select or adapt clustering keys. Availability depends on the table format and runtime.

Aspect	Deletion vectors	Liquid Clustering	CLUSTER BY AUTO
Problem solved	Expensive rewrites for small UPDATE / DELETE / MERGE	Slow reads: poor data skipping	Choosing and maintaining the keys
Effect	Writes are faster; reads apply a small delete bitmap	Reads skip more files	Keys are chosen and adapted for you
Maintenance	`REORG ... APPLY (PURGE)` rewrites files to physically remove rows	`OPTIMIZE` incrementally clusters new data	Requires Predictive Optimization on a managed table
Combine with	Liquid Clustering (yes)	Partitioning / **ZORDER** (no)	Explicit `CLUSTER BY` (replaces it)

When to use it vs. alternatives

- Frequent small MERGE, UPDATE or DELETE operations: evaluate **deletion vectors**.
- Large tables filtered by high-cardinality columns or changing query patterns: evaluate Liquid Clustering.
- Unity Catalog managed tables where you want Databricks to choose layout keys: evaluate CLUSTER BY AUTO.
- Partitioning can still be relevant for some workloads, but Liquid Clustering is the recommended starting point for many new tables.

SQL example: create a table with automatic clustering

```sql
CREATE TABLE main.analytics.orders_clustered (
  order_id BIGINT,
  customer_id BIGINT,
  order_date DATE,
  amount DECIMAL(12, 2)
)
USING DELTA
CLUSTER BY AUTO;
```

Use this only where automatic Liquid Clustering is supported. Check the linked clustering documentation for runtime and managed-table requirements.

Code example: deletion vectors and Liquid Clustering in practice

```sql
-- Deletion vectors: enable per table (default on for new tables in recent runtimes)
ALTER TABLE main.silver.orders SET TBLPROPERTIES ('delta.enableDeletionVectors' = true);

DELETE FROM main.silver.orders WHERE order_id = 42;     -- marks the row, no full file rewrite

-- Physically rewrite files that still contain soft-deleted rows (e.g. for compliance)
REORG TABLE main.silver.orders APPLY (PURGE);

-- Liquid Clustering: explicit keys, changeable at any time without rewriting existing data
CREATE TABLE main.silver.events (event_id BIGINT, user_id BIGINT, event_ts TIMESTAMP, type STRING)
CLUSTER BY (user_id, event_ts);

ALTER TABLE main.silver.events CLUSTER BY (type, event_ts);   -- change keys later
ALTER TABLE main.silver.events CLUSTER BY NONE;               -- turn clustering off
OPTIMIZE main.silver.events;                                   -- clusters data written since the last run
OPTIMIZE main.silver.events FULL;                              -- re-cluster ALL data after changing keys
```

```
PYSPARK / PYTHON
# PySpark: define clustering when writing / creating the table
(spark.table("main.raw.events")
    .write.format("delta")
    .clusterBy("user_id", "event_ts")        # Liquid Clustering from the
DataFrame writer
    .mode("overwrite")
    .saveAsTable("main.silver.events"))
```

Common mistakes

- Thinking deletion vectors replace all file maintenance.
- Combining Liquid Clustering with traditional partitioning or ZORDER on the same table; these approaches are not compatible.
- Assuming CLUSTER BY AUTO is available on every table format/runtime.
- Forgetting that changing clustering keys alone does not necessarily recluster previously written data immediately.

[EXAM TIP] Many small updates/deletes/merges → deletion vectors. Slow selective reads or changing filter columns → Liquid Clustering. "Don't want to choose keys" on a managed table → CLUSTER BY AUTO. Deleted rows are not physically gone until `REORG ... APPLY (PURGE)` and then `VACUUM`.

[PRODUCTION BEST PRACTICE] Choose up to four clustering keys that match the most common filter and join predicates (put the highest-selectivity filter columns first), run `OPTIMIZE` regularly (or let Predictive Optimization do it), and do not cluster on columns you never filter by.

5.3 Caching for repeated reads

Official docs: Disk cache

Core concept

Caching can reduce repeated data-reading work, but Databricks has distinct caching mechanisms. The exam guide calls out "**Delta cache**"; current documentation often calls the underlying capability **disk cache**. Do not confuse disk cache with Spark DataFrame caching or SQL query-result caching.

How it works

Disk cache can retain local copies of remote data files on compute nodes, improving repeated reads when those cached files are reused. It does not mean the SQL result itself is necessarily cached.

When to use it vs. alternatives

Disk cache is relevant for repeated reads of the same underlying files. Spark caching is a separate choice for reusing a DataFrame in a Spark application; query-result caching is a different mechanism for repeated SQL results.

Mechanism	What is cached	Where / scope	Typical trigger
Disk cache (Delta cache)	Remote Parquet/Delta data files, in a fast local format	Local SSD of each worker; survives across queries on that cluster	Automatic on supported instance types or enabled by config
Spark cache (`df.cache()`)	Materialized DataFrame partitions	Executor memory/disk, one application	You call `cache()` / `persist()`
Query result cache	Final result of an identical SQL query	Serverless / SQL warehouse	Automatic for repeated identical queries on unchanged data

SQL example: run a repeated aggregation and compare the query profiles

```sql
SELECT
  vendor_id,
  COUNT(*) AS trip_count,
  AVG(fare_amount) AS avg_fare
FROM samples.nyctaxi.trips
GROUP BY vendor_id;
```

Run the same query more than once and inspect the query profile and elapsed time. This is an experiment, not a command to explicitly enable or guarantee a particular cache.

Code example: configuring and using each cache

```pyspark
# Disk cache (a.k.a. Delta cache): enable on classic compute; automatic on cache-accelerated workers
spark.conf.set("spark.databricks.io.cache.enabled", "true")

# Spark DataFrame cache: reuse an expensive intermediate result within ONE job
dim = spark.table("main.silver.customers").where("active").cache()
dim.count()                    # action materializes the cache
# ... reuse `dim` several times ...
dim.unpersist()                # release memory when finished
```

Common mistakes

- Assuming every query benefits from cache.
- Comparing performance without controlling for warm-up, data changes and compute configuration.
- Calling a cached query result proof that the underlying data scan became more efficient.

> **[EXAM TIP]** Disk cache = cached files on local disk (transparent, no code changes). Spark cache = explicit DataFrame persistence. Result cache = identical SQL results. A cached result tells you nothing about scan efficiency.

5.4 Change Data Feed (CDF) for incremental processing

Official docs: Change Data Feed

Core concept

CDF exposes row-level changes---such as inserts, updates and deletes---so downstream pipelines can process changes rather than repeatedly reprocess a full table.

How it works

Databricks documents legacy CDF for Delta tables and automatic CDF for supported Delta Lake and Apache Iceberg v3 tables. The setup and query syntax differ between these approaches.

When to use it vs. alternatives

Use CDF when downstream consumers need change events or incremental synchronization. Use a full refresh when incremental processing is unsuitable, too complex or not worth the additional logic.

SQL example: enable legacy CDF and read changes

```sql
ALTER TABLE main.analytics.orders
SET TBLPROPERTIES (delta.enableChangeDataFeed = true);

SELECT *
FROM table_changes('main.analytics.orders', 0);
```

This requires an existing Delta table and sufficient privileges. Version 0 is an example starting version; use a valid version for the table and your intended incremental window.

Code example: streaming the change feed into a downstream table

PYSPARK / PYTHON

```
from delta.tables import DeltaTable
from pyspark.sql import Window, functions as F

changes = (spark.readStream
    .option("readChangeFeed", "true")            # emit row-level changes, not
just appends
    .option("startingVersion", 10)               # or startingTimestamp
    .table("main.analytics.orders"))
# Extra columns: _change_type (insert | update_preimage | update_postimage |
delete),
#                _commit_version, _commit_timestamp

def apply_changes(batch_df, batch_id):
    latest = (batch_df.where("_change_type != 'update_preimage'")      # ignore
the "before" image
        .withColumn("rn", F.row_number().over(

Window.partitionBy("order_id").orderBy(F.col("_commit_version").desc())))
        .where("rn = 1"))
    (DeltaTable.forName(spark, "main.gold.orders_mirror").alias("t")
        .merge(latest.alias("s"), "t.order_id = s.order_id")
        .whenMatchedDelete(condition="s._change_type = 'delete'")
        .whenMatchedUpdateAll(condition="s._change_type <> 'delete'")
        .whenNotMatchedInsertAll(condition="s._change_type <> 'delete'")
        .execute())

(changes.writeStream.foreachBatch(apply_changes)
    .option("checkpointLocation",
"/Volumes/main/ops/_checkpoints/orders_mirror")
    .start())
```

SQL

```
-- Batch window: only versions 5 to 8 (inclusive), including deletes
SELECT order_id, _change_type, _commit_version, _commit_timestamp
FROM table_changes('main.analytics.orders', 5, 8)
WHERE _change_type IN ('insert', 'update_postimage', 'delete');
```

Common mistakes

- Assuming deletes and updates are visible from an ordinary append-only stream without change tracking.
- Forgetting the retention window for change history.
- Trying to read change-data files directly rather than using supported table APIs.
- Assuming legacy CDF is automatically enabled for every Delta table.

[EXAM TIP] A plain `readStream` on a Delta table fails (or ignores) upstream updates and deletes - use `readChangeFeed` for row-level changes or `skipChangeCommits` to ignore them. CDF only records changes made after it is enabled, and `VACUUM` removes change data files after retention.

[PRODUCTION BEST PRACTICE] Use CDF for downstream sync, auditing, and propagating deletes (such as GDPR deletes) to derived tables. Consume `update_postimage` and `delete` rows idempotently with MERGE keyed on the primary key and ordered by `_commit_version`.

5.5 Query Profile: find the actual bottleneck

Official docs: Query Profile

Core concept

Query Profile helps you diagnose where SQL execution spends time and resources. The exam guide specifically names poor data skipping, inefficient join strategies and shuffle operations.

How it works

Run a query, open its profile, then inspect operators, row counts, scan metrics, joins and shuffle activity. Use the profile to identify the expensive step before changing the query or table layout.

When to use it vs. alternatives

Use Query Profile for SQL query execution. Use the Spark UI for broader Spark application stages, tasks, executors and streaming diagnostics.

What to look for

Symptom in the profile	Likely cause	Typical remedy
Scan reads most files / bytes despite a selective filter	Poor data skipping (no clustering on the filter column; stale statistics)	Liquid Clustering on filter columns; `ANALYZE TABLE ... COMPUTE STATISTICS`; Predictive Optimization
Large shuffle write before a join	Sort-merge join of a large table with a small one	Broadcast the small side; filter earlier
One task much slower than others	Data skew on the join or group key	AQE skew handling; salting; better key
Spill to disk in aggregate or sort	Not enough memory per task	Reduce data earlier, increase partitions or memory
Many tiny files scanned	Small-file problem	`OPTIMIZE`; auto compaction; clustering
Time spent waiting in queue / compute startup	Warehouse sizing or cold start	Resize or use serverless warehouse

SQL example: a query to profile

```sql
SELECT
  t.vendor_id,
  COUNT(*) AS trip_count,
  SUM(t.fare_amount) AS total_fares
FROM samples.nyctaxi.trips AS t
WHERE t.tpep_pickup_datetime >= TIMESTAMP '2016-01-01 00:00:00'
  AND t.tpep_pickup_datetime <  TIMESTAMP '2016-01-02 00:00:00'
GROUP BY t.vendor_id;
```

Open the query profile after running it. The SQL is illustrative; the profile is where you verify the bottleneck.

Code example: act on what the profile shows

```sql
-- Inspect the plan without running the query
EXPLAIN FORMATTED
SELECT /*+ BROADCAST(c) */ o.order_id, c.region      -- force a broadcast join for a small dimension
FROM main.silver.orders o JOIN main.silver.customers c ON o.customer_id = c.customer_id
WHERE o.order_date >= '2026-10-01';

-- Refresh stats so the optimizer can choose join order and file pruning
ANALYZE TABLE main.silver.orders COMPUTE STATISTICS FOR COLUMNS customer_id, order_date;
```

```python
# Adaptive Query Execution is on by default on Databricks; these are the knobs worth knowing
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 50 * 1024 * 1024)   # 50 MB
spark.conf.set("spark.sql.shuffle.partitions", "auto")                      # let AQE size shuffles
```

Common mistakes

- Optimizing a query from wall-clock time alone.
- Assuming a join is inefficient without checking the plan and input sizes.
- Applying clustering or caching without verifying that scans or shuffles are the bottleneck.

> **[EXAM TIP]** Read the profile first: files read vs. files pruned tells you about data skipping; the shuffle/exchange operators tell you about join strategy; one straggler task tells you about skew. Do not resize the cluster until you know which one it is.

5.6 Liquid Clustering vs. partitioning and ZORDER

Official docs: Liquid Clustering

Core concept

The exam expects you to choose a table layout based on table size, data distribution and query patterns---not memorize one universally best strategy.

How it works

Liquid Clustering organizes data by clustering keys and lets those keys change without the rigid partition layout of traditional partitioning. It is intended to replace partitioning and ZORDER for many supported use cases.

Z-Ordering vs. Liquid Clustering vs. partitioning

Aspect	Partitioning	ZORDER	Liquid Clustering
Layout defined by	Directory per partition value	`OPTIMIZE ... ZORDER BY` multi-dimensional sort	`CLUSTER BY` keys, maintained incrementally
Changing keys	Requires rewriting the table	Specify different columns on each **OPTIMIZE** (full re-sort of touched files)	`ALTER TABLE ... CLUSTER BY` with no rewrite of existing data
Cardinality sweet spot	Low (days, regions); partitions of at least ~1 GB	Moderate to high	High cardinality works well
Maintenance cost	None, but small-file risk	Re-sorts files each run	Incremental, only unclustered data
Combine with others	With ZORDER	With partitioning	Not with partitioning or ZORDER
Recommended for new tables	Rarely	Legacy	Yes, default choice

When to use it vs. alternatives

- Liquid Clustering: changing query patterns, high-cardinality filters, skewed or fast-growing tables.
- Partitioning: consider when a workload and data layout genuinely benefit from partition pruning and partition sizes remain sensible.
- ZORDER: recognize the older optimization technique, but avoid combining it with Liquid Clustering on the same table.

SQL example: explicit clustering

```sql
CREATE TABLE main.analytics.orders_by_customer
USING DELTA
CLUSTER BY (customer_id, order_date)
AS
SELECT order_id, customer_id, order_date, amount
FROM main.raw.orders;
```

Choose keys based on observed query patterns. The table and source must exist in a supported environment, and your identity must have the necessary privileges.

Code example: legacy layouts and migration

```sql
-- Legacy: partitioned table with ZORDER (recognize it on the exam)
CREATE TABLE main.legacy.events_part (event_id BIGINT, event_date DATE, user_id
BIGINT)
PARTITIONED BY (event_date);
OPTIMIZE main.legacy.events_part ZORDER BY (user_id);

-- Migrate to Liquid Clustering: you cannot ALTER a partitioned table to
CLUSTER BY, so rebuild it
CREATE TABLE main.silver.events_lc
CLUSTER BY (user_id, event_date)
AS SELECT * FROM main.legacy.events_part;
```

Common mistakes

- Creating many small partitions.
- Choosing clustering columns based on intuition instead of actual query predicates.
- Expecting the data layout to improve before the relevant optimization work occurs.

> **[EXAM TIP]** Changing filter columns, high-cardinality keys, or fast-growing skewed tables → Liquid Clustering. Never `PARTITIONED BY` or `ZORDER` together with `CLUSTER BY`. Partition only when each partition is large (about 1 GB or more) and queries filter on the partition column.

> **[PRODUCTION BEST PRACTICE]** New table? Start with `CLUSTER BY` (or `CLUSTER BY AUTO` on a managed table), let Predictive Optimization run `OPTIMIZE`, and revisit keys quarterly against real query predicates from `system.query.history`.

Section 6 — Ensuring Data Security and Compliance (8%)

6.1 Least-privilege access control

Official docs: Unity Catalog permissions model · Access control overview

Core concept

Grant each principal only the privileges needed for its work. In Unity Catalog, data access typically requires the relevant usage privileges on the parent catalog and schema plus the privilege on the target object.

How it works

Unity Catalog controls access to securable objects such as catalogs, schemas, tables, views and functions. Workspace objects have their own access-control model.

Privilege	Purpose
USE CATALOG / USE SCHEMA	Traverse into the parent container; required but not sufficient to read data
SELECT	Read a table, view, or materialized view
MODIFY	INSERT, UPDATE, DELETE, MERGE on a table
EXECUTE	Call a function or use a model
READ VOLUME / WRITE VOLUME	Read or write files in a volume
CREATE TABLE / CREATE SCHEMA	Create objects inside a schema or catalog
MANAGE	Manage privileges and ownership on an object without being the owner

When to use it vs. alternatives

Use Unity Catalog grants for data objects and workspace permissions for workspace resources. Prefer group-based grants over managing permissions separately for every individual user.

SQL example: grant a group read access to one table

```sql
GRANT USE CATALOG ON CATALOG main TO `analysts`;
GRANT USE SCHEMA ON SCHEMA main.analytics TO `analysts`;
GRANT SELECT ON TABLE main.analytics.customers TO `analysts`;
```

Run these statements as a principal authorized to grant the privileges. Replace the catalog, schema, table and group with your actual objects.

Code example: service principals, verification, and revocation

```sql
-- Pipelines run as a service principal that can write only to its own schema
GRANT USE CATALOG ON CATALOG main TO `sp-etl-prod`;
GRANT USE SCHEMA, CREATE TABLE, MODIFY, SELECT ON SCHEMA main.silver TO `sp-etl-prod`;
GRANT READ VOLUME ON VOLUME main.raw.landing TO `sp-etl-prod`;

-- Verify exactly what a principal can do (including inherited grants)
SHOW GRANTS `analysts` ON TABLE main.analytics.customers;
SHOW GRANTS ON SCHEMA main.analytics;

-- Remove access when no longer needed
REVOKE SELECT ON TABLE main.analytics.customers FROM `contractors`;

-- Ownership: transfer to a group so objects survive staff changes
ALTER TABLE main.analytics.customers OWNER TO `data-platform-admins`;
```

Common mistakes

- Granting SELECT on a table but forgetting `USE CATALOG` and `USE SCHEMA`.
- Granting broad privileges such as `ALL PRIVILEGES` when only read access is required.
- Assuming a user's ownership of a parent object means they own every child object.

> **[EXAM TIP]** "User gets permission denied despite SELECT on the table" → check `USE CATALOG` and `USE SCHEMA` on the parents. Least privilege means groups, narrow scope, and service principals for automation.

6.2 Attribute-based access control (ABAC), governed tags, row filters and column masks

Official docs: ABAC core concepts · Row filters and column masks · ABAC configuration tutorial

Core concept

ABAC uses **governed tags** and centrally defined policies to apply access rules to matching objects. This is particularly important in the new guide: it explicitly tests using ABAC to enforce row filters and column masks at scale.

How it works

- A **row filter** restricts which rows a user can see.
- A **column mask** controls the value returned for a sensitive column.
- An ABAC policy can apply these controls based on governed tags, reducing the need to configure each table separately.

When to use it vs. alternatives

Use table-level row filters and masks for a small number of table-specific controls. Prefer ABAC for consistent, tag-driven policies across many tables. Dynamic views are another option when you need a governed presentation layer or a custom projection.

Mechanism	Scope	Configured	Best for
Row filter / column mask (table-level)	One table or column	`ALTER TABLE ... SET ROW FILTER / SET MASK` per table	A few table-specific controls
ABAC policy	Every object matching a governed tag, present and future	One policy at catalog or schema level	Consistent controls across hundreds of tables
Dynamic view	One view definition	`CREATE VIEW ... CASE WHEN is_account_group_member(...)`	Custom projections, joins, or when the base table must stay hidden

SQL example: create a masking function and apply it to one column

```sql
CREATE OR REPLACE FUNCTION main.analytics.mask_email(email STRING)
RETURNS STRING
RETURN '***MASKED***';

ALTER TABLE main.analytics.customers
ALTER COLUMN email
SET MASK main.analytics.mask_email;
```

This is a table-level mask example, not a complete ABAC policy. It requires the table and schema to exist and appropriate privileges. For a tag-driven policy that scales across tables, follow the official ABAC tutorial linked above.

Code example: group-aware mask, row filter, and tag-driven policy

```sql
-- Column mask that depends on who is asking
CREATE OR REPLACE FUNCTION main.governance.mask_email(email STRING)
RETURNS STRING
RETURN CASE WHEN is_account_group_member('pii_readers') THEN email
            ELSE regexp_replace(email, '^[^@]+', '***') END;     -- keep only
the domain

ALTER TABLE main.analytics.customers ALTER COLUMN email SET MASK
main.governance.mask_email;

-- Row filter: regional managers only see their own region
CREATE OR REPLACE FUNCTION main.governance.region_filter(region STRING)
RETURN is_account_group_member('global_analysts')
    OR is_account_group_member(CONCAT('region_', lower(region)));

ALTER TABLE main.analytics.customers SET ROW FILTER
main.governance.region_filter ON (region);

-- Tag-driven ABAC (syntax per the ABAC docs; verify availability in your
workspace)
SET TAG ON COLUMN main.analytics.customers.email pii = 'email';   -- governed
tag assignment

CREATE POLICY mask_pii_email ON CATALOG main
COLUMN MASK main.governance.mask_email
TO `analysts` EXCEPT `pii_readers`
FOR TABLES
MATCH COLUMNS hasTagValue('pii', 'email') AS email_col
ON COLUMN email_col;                                                 -- applies
to EVERY matching column
```

Common mistakes

- Treating a tag as a security policy by itself; a policy must define the control.
- Applying a table-level mask to one table when the requirement covers hundreds of current and future tables.
- Assuming a masked result means the underlying sensitive data has been deleted.
- Ignoring the privileges needed to manage governed tags and policies.

[EXAM TIP] Hundreds of tables, new tables added constantly, centrally governed → ABAC with governed tags. One table → a direct row filter or column mask. A tag classifies; a policy enforces. Masking changes what a reader sees, not what is stored.

> **[PRODUCTION BEST PRACTICE]** Define a small taxonomy of governed tags (for example `pii`, `sensitivity`), restrict who can assign them, write ABAC policies once at catalog level, and combine with Data Classification to tag new columns automatically. Keep exception groups (`EXCEPT`) tiny and audited.

6.3 Anonymization and pseudonymization

Official docs: Row filters and column masks · SQL sha2 function

Core concept

These techniques reduce exposure of confidential data, but they are not interchangeable.

Technique	What it does	Reversible? / risk
Hashing	Transforms a value into a digest	Predictable inputs can still be guessed (dictionary attack); use a secret salt or HMAC
Tokenization	Replaces a value with a token	Reversible only via a separately controlled mapping or service
Suppression	Removes or withholds a value	Irreversible
Generalization	Replaces precise information with a broader category, such as an age band	Irreversible; may reduce utility

How it works

Implement the transformation in a controlled pipeline or apply access-time masking where appropriate. Unity Catalog masks can limit what a user sees, but a mask does not by itself permanently anonymize the stored data.

When to use it vs. alternatives

Use pseudonymization when downstream processing still needs a stable identifier. Use suppression or generalization when exact values are not required. Choose tokenization when a controlled token service or mapping is part of the architecture.

SQL example: pseudonymize and generalize values in a result

```sql
SELECT
  sha2(email, 256) AS pseudonymous_email,
  CASE
    WHEN age BETWEEN 20 AND 29 THEN '20-29'
    WHEN age BETWEEN 30 AND 39 THEN '30-39'
    ELSE 'Other'
  END AS age_band,
  CAST(NULL AS STRING) AS suppressed_email
FROM main.analytics.customers;
```

This is an illustrative transformation. It assumes email and age exist and does not constitute a complete privacy guarantee. For real deployments, also consider access to source tables, the re-identification risk and the applicable policy.

Code example: salted hashing and a tokenization pattern

SQL

```sql
-- Salted hash: the salt lives in a secret scope, so a dictionary attack needs the secret
SELECT sha2(CONCAT(lower(trim(email)), secret('privacy', 'email_salt')), 256) AS email_token
FROM main.raw.customers;
```

PYSPARK / PYTHON

```python
from pyspark.sql import functions as F

salt = dbutils.secrets.get("privacy", "email_salt")

pseudo = (spark.table("main.raw.customers")
    .withColumn("customer_key", F.sha2(F.concat(F.lower(F.trim("email")), F.lit(salt)), 256))
    .withColumn("zip3", F.substring("zip", 1, 3))          # generalization: 5-digit zip -> 3 digits
    .drop("email", "zip", "phone"))                         # suppression: remove direct identifiers

# Tokenization: a separate, tightly governed mapping table holds token -> original value
# main.vault.email_tokens is readable ONLY by a small privileged group
```

Common mistakes

- Calling a plain hash "anonymous" without considering re-identification or dictionary attacks.
- Using a deterministic hash of an email as if it could never be linked back to a person.
- Treating a query-time mask as a replacement for storage-level deletion.

> **[EXAM TIP]** Hash ≠ anonymous. Pseudonymized data is still personal data under regulations such as GDPR. Anonymization requires that re-identification is no longer reasonably possible.

6.4 Detect and mask PII in batch and streaming pipelines

Official docs: Data Classification · ABAC core concepts

Core concept

Sensitive-data protection needs both discovery and enforcement. Databricks Data Classification can help identify sensitive columns; Unity Catalog tags and ABAC can then support governance controls. Pipelines must also ensure sensitive values are handled correctly as records arrive.

How it works

Data Classification scans catalog data and can tag detections. It runs as a background process; the docs say new tables and columns are typically scanned within 24 hours. That means it is not a synchronous per-record PII detector for a streaming pipeline. Use explicit pipeline transformation or validation logic when a record must be masked before it is written to a protected target.

When to use it vs. alternatives

Use Data Classification to discover and govern sensitive data across catalogs. Use deterministic, tested pipeline logic for immediate batch/streaming enforcement. Use column masks for query-time protection of users who should not see raw values.

SQL example: mask a sensitive field as part of a transformation

```sql
CREATE OR REPLACE TABLE main.analytics.customers_masked AS
SELECT
  customer_id,
  '***MASKED***' AS email,
  region
FROM main.raw.customers;
```

This example illustrates a batch transformation. For streaming, implement the same approved logic in the streaming pipeline definition and verify that raw-source access is restricted. A production design should also specify whether the original data may be retained and who can access it.

Code example: PII masking in a streaming pipeline

PYSPARK / PYTHON

```
from pyspark import pipelines as dp
from pyspark.sql import functions as F

EMAIL = r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}"
SSN   = r"\b\d{3}-\d{2}-\d{4}\b"

@dp.table(name="support_tickets_masked", comment="PII removed before landing in
silver")
@dp.expect_or_drop("no_ssn_left", "NOT (ticket_text RLIKE '\\\\b\\\\d{3}-
\\\\d{2}-\\\\d{4}\\\\b')")  # safety net
def support_tickets_masked():
    return (spark.readStream.table("bronze_support_tickets")
        .withColumn("ticket_text", F.regexp_replace("ticket_text", EMAIL,
"[EMAIL]"))
        .withColumn("ticket_text", F.regexp_replace("ticket_text", SSN,
"[SSN]"))
        .drop("customer_email"))                    # never propagate the raw
identifier
```

Common mistakes

- Assuming a background classification scan will mask a new record before it lands.
- Protecting only dashboards while leaving raw tables broadly accessible.
- Failing to test both batch and streaming paths.
- Giving users access to classification result tables that contain sensitive sample values.

> **[EXAM TIP]** Discovery (Data Classification, asynchronous) and enforcement (pipeline logic, masks, ABAC) are different jobs. A question that needs masking before the record lands in a table is answered by pipeline transformation, not by a background scan.

6.5 Data purging and retention compliance

Official docs: GDPR data handling in Lakeflow pipelines · Delta VACUUM

Core concept

A compliant purge removes records according to a retention or deletion policy and considers the different places where data can remain, including table history and obsolete data files.

How it works

Delta `DELETE` changes the logical table contents. Physical removal of unreferenced files is a separate concern governed by retention and `VACUUM` behavior. Some configurations and features can affect when data is physically removed.

Layer	Where deleted data can still exist	What removes it
Current table version	Rows present	`DELETE` / `MERGE ... WHEN MATCHED DELETE`
Data files with deletion vectors	File still contains the soft-deleted row	`REORG TABLE ... APPLY (PURGE)`
Older table versions (time travel)	Files referenced by earlier versions	`VACUUM` after the retention window
Change Data Feed files	`_change_data` files hold before-images	`VACUUM` after retention
Downstream copies	Silver/gold tables, exports, caches, shares	Propagate the delete (CDF, pipelines) and clean those systems

When to use it vs. alternatives

Use a targeted DELETE for record-level deletion. Use a documented retention workflow to coordinate logical deletion, physical cleanup and any required downstream deletion. Do not shorten retention simply to make data disappear faster without checking safety and recovery requirements.

SQL example: delete records past an approved retention date

```sql
DELETE FROM main.analytics.customers
WHERE deletion_requested = true;
```

This assumes the table has a `deletion_requested` column and that the predicate exactly represents approved deletion requests. Before implementing physical cleanup, validate your retention requirements and follow the official VACUUM documentation.

Code example: end-to-end purge workflow

The original guide intentionally avoided a destructive `VACUUM` default. The workflow below shows the safe order of operations: delete, purge soft-deleted rows, preview with `DRY RUN`, then vacuum only after confirming retention requirements.

```sql
-- 1) Logical delete
DELETE FROM main.analytics.customers WHERE customer_id IN (SELECT customer_id
FROM main.ops.erasure_requests);

-- 2) If deletion vectors are enabled, rewrite files that still physically
contain the rows
REORG TABLE main.analytics.customers APPLY (PURGE);

-- 3) Preview which files VACUUM would remove (safe: nothing is deleted)
VACUUM main.analytics.customers RETAIN 168 HOURS DRY RUN;

-- 4) Physically remove unreferenced files older than the retention window
(default 7 days)
VACUUM main.analytics.customers;

-- Inspect history and retention-related properties
DESCRIBE HISTORY main.analytics.customers;
ALTER TABLE main.analytics.customers SET TBLPROPERTIES (
  'delta.deletedFileRetentionDuration' = 'interval 7 days',   -- how long
removed files are kept
  'delta.logRetentionDuration'         = 'interval 30 days'); -- how long
history is kept
```

Common mistakes

- Assuming DELETE always immediately removes every physical copy.
- Running **VACUUM** without understanding its effect on time travel and recovery.
- Forgetting copies in downstream tables, exports, caches or other systems.
- Treating one SQL statement as proof of regulatory compliance.

[EXAM TIP] DELETE is logical; VACUUM is physical. After VACUUM you lose time travel to the removed versions. Reducing retention below the default (7 days) is blocked unless safety checks are overridden - and doing so can break concurrent readers and streams.

[PRODUCTION BEST PRACTICE] Automate erasure requests as a pipeline: delete in the source of truth, propagate to derived tables (CDF or AUTO CDC deletes), run `REORG ... PURGE` and `VACUUM` on a schedule that meets the legal deadline, and keep an audit record (`DESCRIBE HISTORY`, `system.access.audit`).

Section 7 — Data Governance (5%)

7.1 Tags and comments for data discoverability

Official docs: Apply tags to Unity Catalog securable objects · Unity Catalog data discovery

Core concept

Metadata helps people find and understand data assets. Comments describe an object or column in human-readable terms; tags categorize objects using key-value attributes.

How it works

Unity Catalog supports comments and tags on supported securable objects. Tags can also support search and can be used by ABAC policies. Tag values are stored as plain text, so do not put sensitive information in them.

When to use it vs. alternatives

Use comments to explain meaning, ownership context or intended use. Use tags for consistent classification, such as `data_domain`, `environment` or governed sensitivity categories. Use governed tags when assignment needs centrally controlled allowed values and permissions.

Metadata	Purpose	Notes
Comment	Human-readable meaning of a table or column	Written for people and search; also read by AI assistants
Tag (free-form)	Key-value classification for search and filtering	Anyone with permission can set any value: risk of inconsistency
Governed tag	Tag with centrally defined allowed values and controlled assignment	Supports ABAC policies and consistent taxonomy

SQL example

```sql
COMMENT ON TABLE main.analytics.customers
IS 'Customer analytics table for approved internal reporting';

COMMENT ON COLUMN main.analytics.customers.customer_id
IS 'Unique identifier assigned to each customer';

ALTER TABLE main.analytics.customers
SET TAGS ('data_domain' = 'customer');
```

This assumes the table and column exist and that you have permission to update metadata and tags.

Code example: column tags and discovery queries

```sql
-- Column-level tag
ALTER TABLE main.analytics.customers ALTER COLUMN email SET TAGS ('pii' =
'email');

-- Find every table in the domain (information_schema exposes the metadata)
SELECT catalog_name, schema_name, table_name, tag_name, tag_value
FROM main.information_schema.table_tags
WHERE tag_name = 'data_domain' AND tag_value = 'customer';

SELECT table_name, column_name, tag_name, tag_value
FROM main.information_schema.column_tags
WHERE tag_name = 'pii';

-- Remove an outdated tag
ALTER TABLE main.analytics.customers UNSET TAGS ('data_domain');
```

Common mistakes

- Using inconsistent free-form tags that mean the same thing.
- Putting PII or other sensitive information in tag values.
- Assuming tags automatically grant access or mask data without a policy.
- Writing comments that describe implementation details but not business meaning.

> **[EXAM TIP]** Comment = meaning; tag = classification; governed tag = classification with enforced vocabulary and permissions. Tags alone never restrict access.

7.2 Unity Catalog permission inheritance

Official docs: Unity Catalog permissions model concepts

Core concept

Unity Catalog privileges can inherit downward from catalogs and schemas to their child objects. This reduces repeated grants and is an important exam distinction.

How it works

- A privilege granted on a catalog can apply to current and future child schemas and their objects.
- A privilege granted on a schema can apply to its tables, views, volumes and functions.
- Users still need the required usage privileges on the parent catalog and schema for ordinary data access.
- Ownership does not simply inherit downward as ownership of every child.

When to use it vs. alternatives

Use inherited grants for consistent access across a domain or schema. Use object-level grants when only specific tables should be accessible. Be cautious with broad catalog-level grants because they can apply to future objects as well.

SQL example — grant read access across a schema

```
SQL
GRANT USE CATALOG ON CATALOG main TO `analysts`;
GRANT USE SCHEMA ON SCHEMA main.analytics TO `analysts`;
GRANT SELECT ON SCHEMA main.analytics TO `analysts`;
```

The last grant is broader than granting SELECT on one table: it is intended to cover the schema's objects through inheritance. Confirm the exact privilege support for your Unity Catalog privilege model and make sure this broad scope is intended before running it.

Code example: verifying and limiting inheritance

```
SQL
-- Effective grants include inherited ones
SHOW GRANTS ON SCHEMA main.analytics;
SHOW GRANTS `analysts` ON TABLE main.analytics.new_table_created_today;   --
shows inherited SELECT

-- Carve-out pattern: do NOT grant at schema level; grant per table instead
REVOKE SELECT ON SCHEMA main.analytics FROM `contractors`;
GRANT  SELECT ON TABLE main.analytics.public_metrics TO `contractors`;

-- Broad catalog-level read for a trusted platform group (applies to FUTURE
schemas too)
GRANT SELECT ON CATALOG main TO `data-platform-admins`;
```

Common mistakes

- Thinking a grant applies only to objects that existed when it was issued.
- Assuming SELECT alone is enough when parent usage privileges are missing.
- Confusing inherited privileges with inherited ownership.
- Granting at catalog level without considering future tables.

[EXAM TIP] Future objects inherit parent grants automatically - a catalog-level `SELECT` exposes tables created next month. Inheritance covers privileges, not ownership.

[PRODUCTION BEST PRACTICE] Grant at the schema level to groups aligned with a data domain, reserve catalog-level grants for platform and admin groups, and review `SHOW GRANTS` on each schema as part of release checks.

Section 8 — Debugging and Deploying (10%)

8.1 Diagnose failures with Spark UI, logs, system tables and Query Profile

Official docs: Debugging with the Spark UI · System tables · Query Profile · Pipeline event log schema · Jobs system table reference

Core concept

Debugging starts by identifying where the failure occurs. The exam expects you to select the right diagnostic source for the symptom rather than rely on a single tool.

How it works on Databricks

Diagnostic source	Best use
Spark UI	Stages, tasks, shuffles, skew, executor activity and streaming execution
Driver/executor logs	Exceptions, startup failures, dependency problems and runtime errors
System tables	Historical job, compute, billing and audit analysis
Query Profile	SQL operators, scan efficiency, joins and query bottlenecks
Pipeline event log	Pipeline progress, data-quality events, lineage and pipeline errors

Spark UI symptom guide

What you see	Meaning	Typical fix
One task takes far longer than the rest in a stage	Data skew	Salting, AQE skew join, repartition on a better key
Shuffle spill (memory/disk) is large	Partitions too big for task memory	More shuffle partitions, filter earlier, bigger workers
Many short tasks with scheduling delay	Too many small partitions/files	`OPTIMIZE`, coalesce, auto compaction
`OutOfMemoryError` on the driver	`collect()`, large broadcast, or big `toPandas()`	Avoid collecting; aggregate first; raise driver memory
`OutOfMemoryError` on executors	Skew or oversized partitions	Repartition, avoid exploding without filters
Streaming: input rate above processing rate	Falling behind	Rate-limit, scale compute, optimize the sink

When to use it vs. alternatives

- If a task fails immediately, inspect the error and driver logs first.
- If a task runs slowly, inspect Spark stages or Query Profile, depending on whether it is a Spark application or SQL workload.
- If failures recur across jobs or days, use system tables to identify patterns.
- If the pipeline fails a quality expectation, start with its event log and data-quality details.

SQL example — find recent failed job runs

```sql
SELECT
  workspace_id,
  job_id,
  run_id,
  result_state,
  period_start_time,
  period_end_time
FROM system.lakeflow.job_run_timeline
WHERE result_state = 'FAILED'
ORDER BY period_start_time DESC
LIMIT 50;
```

Use this to identify runs worth investigating. The table's availability, schema and permissions depend on your environment; consult the Jobs system table reference.

Code example: recurring failures and termination codes

```sql
-- Which jobs fail repeatedly, and with what termination code?
SELECT job_id, termination_code, COUNT(DISTINCT run_id) AS failed_runs
FROM system.lakeflow.job_run_timeline
WHERE result_state IN ('FAILED', 'ERROR', 'TIMED_OUT')
  AND period_start_time >= CURRENT_TIMESTAMP() - INTERVAL 14 DAYS
GROUP BY job_id, termination_code
ORDER BY failed_runs DESC;
```

```python
# Distinguish a data problem from a code problem quickly in a notebook
try:
    df = spark.table("main.silver.orders")
    df.select("amount").where("amount < 0").limit(1).collect()
except Exception as e:
    print(type(e).__name__, str(e)[:500])   # read the FIRST error, not just the last line of the stack
```

Common mistakes

- Increasing compute before identifying the bottleneck.
- Confusing a timeout with an application exception.
- Looking only at the final error message instead of the earliest meaningful failure.
- Treating query slowness and pipeline orchestration failure as the same problem.

> **[EXAM TIP]** Match symptom to tool: slow SQL → Query Profile; slow Spark stage/skew/spill → Spark UI; task crashes → driver/executor logs; recurring across days → system tables; expectation or flow failure in a pipeline → event log.

8.2 Repair failed jobs and override parameters

Official docs: Repair job failures · Monitor Lakeflow Jobs

Core concept

A repair lets you rerun failed or otherwise eligible parts of a job run instead of blindly rerunning the entire workflow. Parameter overrides let you correct run inputs when a failure was caused by an incorrect parameter.

How it works

Inspect the failed run, identify the failed task and its upstream dependencies, determine whether the cause is transient or requires a fix, then use the supported repair workflow.

When to use it vs. alternatives

- Repair a run when the relevant tasks can be safely rerun and the cause has been addressed.
- Rerun the full job when the workflow's correctness requires it or a repair is not appropriate.
- Fix the code, permissions, dependencies or source data before retrying when the error is deterministic.

Aspect	Repair run	New run (run now)
Scope	Failed tasks and the tasks that depend on them	The entire job
Completed tasks	Kept (not re-executed)	Executed again
Run history	Same run id, extra repair attempt	New run id
Cost / time	Lower	Higher
Use when	A subset failed and reruns are safe	State must be rebuilt from scratch

Runnable Python example — inspect recent runs for a specific job

PYSPARK / PYTHON

```
from databricks.sdk import WorkspaceClient

w = WorkspaceClient()
job_id = 12345  # Replace with your job ID.

for run in w.jobs.list_runs(job_id=job_id, limit=10):
    state = run.state
    lifecycle = state.life_cycle_state if state else None
    result = state.result_state if state else None
    print(
        f"run_id={run.run_id}, "
        f"lifecycle={lifecycle}, result={result}"
    )
```

This example lists recent runs; it does not perform a repair. Use the official repair documentation for the supported UI/API workflow and the repair parameters relevant to your job.

Code example: performing a repair with overridden parameters

PYSPARK / PYTHON

```
# Repair only the failed tasks of run 987654321 and correct a bad parameter
w.jobs.repair_run(
    run_id=987654321,
    rerun_all_failed_tasks=True,                    # or
rerun_tasks=["transform", "publish"]
    job_parameters={"run_date": "2026-10-23"},     # override the parameter
that caused the failure
).result()
```

CLI / SHELL

```
databricks jobs repair-run --json '{
  "run_id": 987654321,
  "rerun_tasks": ["transform"],
  "job_parameters": {"run_date": "2026-10-23"}
}'
```

Common mistakes

- Repeating a repair without changing a bad input or correcting the underlying error.
- Assuming every failed task can safely be rerun without side effects.
- Changing a parameter without checking downstream tasks' expectations.
- Treating a repair as a substitute for root-cause analysis.

> **[EXAM TIP]** Repair keeps successful tasks and reruns failed ones plus dependents - cheaper than a full rerun. Deterministic failures (bad code, missing permission) will fail again; fix first, then repair. Idempotency decides whether rerunning is safe.

8.3 Debug Lakeflow pipelines and Spark workloads with event logs and Spark UI

Official docs: Monitor pipeline event logs · Debugging with the Spark UI

Core concept

Pipeline event logs provide pipeline-level context; the Spark UI and logs provide execution-level detail. Use them together when you need to trace a pipeline failure from its reported event to the underlying Spark operation.

How it works

1. Find the failed pipeline update and the earliest relevant error event.
2. Inspect the event's message and structured details.

3. Identify the affected flow, table or operation.
4. Open relevant Spark execution information and logs.
5. Fix the root cause, then rerun or repair as appropriate.

When to use it vs. alternatives

Use event logs for pipeline progress and quality events; use the Spark UI when the failure relates to task execution, shuffle, memory or executor behavior.

SQL example — inspect the most recent pipeline events

```sql
SELECT
  timestamp,
  event_type,
  level,
  message,
  details
FROM event_log('YOUR-PIPELINE-ID')
WHERE level IN ('WARN', 'ERROR')
ORDER BY timestamp DESC
LIMIT 100;
```

Replace the placeholder with the pipeline ID and run in a supported context with access to the pipeline event log.

Code example: trace a failing flow

```sql
-- 1) Earliest error of the latest update (root cause, not the cascade)
SELECT timestamp, origin.flow_name, message,
       details:flow_progress.status AS status, error.exceptions[0].message AS
root_exception
FROM event_log('YOUR-PIPELINE-ID')
WHERE level = 'ERROR'
ORDER BY timestamp ASC
LIMIT 5;

-- 2) Which flows failed in the last update?
SELECT origin.flow_name, details:flow_progress.status AS status
FROM event_log('YOUR-PIPELINE-ID')
WHERE event_type = 'flow_progress' AND details:flow_progress.status IN
('FAILED', 'STOPPING');
```

```
CLI / SHELL
# Re-run after the fix: refresh only the failed tables instead of the whole pipeline
databricks pipelines start-update <pipeline-id> --refresh-selection orders_clean
# Full refresh of a streaming table re-reads the source and TRUNCATES the target - use with care
databricks pipelines start-update <pipeline-id> --full-refresh-selection orders_clean
```

Common mistakes

- Assuming every pipeline failure is a Spark performance problem.
- Ignoring the structured event details.
- Making changes before identifying which update and flow failed.

> **[EXAM TIP]** Start at the pipeline event log for "what failed and why", move to the Spark UI for "why was it slow or out of memory". Many pipeline failures are data-quality (`expect_or_fail`), schema, or permission problems, not Spark tuning problems.

8.4 Deploy resources with Declarative Automation Bundles

Official docs: What are Declarative Automation Bundles? · Deploy a bundle

Core concept

Declarative Automation Bundles (formerly Databricks Asset Bundles) define a project's source files, configuration, jobs, pipelines and related resources as deployable source-controlled assets.

How it works

A bundle uses configuration files---typically including `databricks.yml`---to describe resources and deployment targets. The CLI validates and deploys the project to an environment such as development, staging or production.

When to use it vs. alternatives

Use bundles for repeatable, source-controlled deployments and CI/CD. Manual UI creation may be useful for exploration, but by itself it does not give the same reproducible deployment workflow.

Runnable CLI example — run from the root of a correctly configured bundle project

```
CLI / SHELL
databricks bundle validate -t dev
databricks bundle deploy -t dev
```

This assumes the Databricks CLI is installed, authenticated and compatible with the bundle project, and that a `dev` target is defined. These commands are not standalone: they need the bundle's source and configuration files.

Code example: a complete multi-target bundle with a pipeline and a job

```yaml
# databricks.yml
bundle:
  name: orders_platform

include:
  - resources/*.yml

variables:
  catalog:     { default: dev }
  alert_email: { default: dev-team@example.com }

targets:
  dev:
    mode: development
    default: true
    workspace: { host: https://dev.cloud.databricks.com }
  staging:
    workspace: { host: https://stg.cloud.databricks.com }
    variables: { catalog: staging }
  prod:
    mode: production
    workspace:
      host: https://prod.cloud.databricks.com
      root_path: /Workspace/Shared/.bundle/${bundle.name}/${bundle.target}
    variables: { catalog: prod, alert_email: data-oncall@example.com }
    run_as: { service_principal_name: 00000000-0000-0000-0000-000000000000 }
    permissions:
      - group_name: data-engineers
        level: CAN_VIEW
```

YAML

```yaml
# resources/pipelines.yml and resources/jobs.yml
resources:
  pipelines:
    orders_pipeline:
      name: orders_pipeline_${bundle.target}
      catalog: ${var.catalog}
      schema: silver
      serverless: true
      libraries:
        - file: { path: ../src/pipelines/orders.py }

  jobs:
    orders_daily:
      name: orders_daily_${bundle.target}
      email_notifications: { on_failure: ["${var.alert_email}"] }
      tasks:
        - task_key: refresh_pipeline
          pipeline_task: { pipeline_id:
${resources.pipelines.orders_pipeline.id} }
```

CLI / SHELL

```
databricks bundle validate -t prod              # catches schema and reference
errors
databricks bundle deploy   -t prod              # idempotent: creates or
updates resources
databricks bundle run orders_daily -t prod      # smoke test the deployed job
databricks bundle deployment bind orders_daily 123456789 -t prod   # adopt an
existing job into the bundle
```

Common mistakes

- Editing a bundle-managed job in the UI and assuming the source configuration changes too.
- Deploying to the wrong target.
- Skipping validation.
- Keeping environment-specific values hardcoded instead of using target configuration.

[EXAM TIP] Promotion = same code, different `-t target`. `mode: development` prefixes names and pauses schedules; `mode: production` expects explicit paths and a non-personal identity. UI edits to bundle-managed resources create drift that the next deploy overwrites.

8.5 Git-based CI/CD and Databricks Git folders

Official docs: Databricks Git folders · Declarative Automation Bundles

Core concept

Git provides version control and collaboration; Databricks **Git folders** (formerly Repos) let you work with Git-backed code in the workspace. CI/CD automates checks and deployment after changes are committed or merged.

How it works

A typical workflow is to make a change in a branch, commit and push it, run tests and validation, and deploy through a configured pipeline---often using Declarative Automation Bundles.

When to use it vs. alternatives

Use Git folders for Git-backed development in the workspace. Use a CI/CD workflow and bundles when you need automated, consistent promotion across environments. A Git push alone does not deploy resources unless deployment automation is configured.

Aspect	Git folders	Bundles + CI/CD
Purpose	Develop and version code in the workspace	Deploy jobs, pipelines and code consistently across environments
Triggers deployment?	No	Yes, when invoked by the CI system
Typical environment	Dev (feature branches)	Staging and production

Runnable Git example — standard local Git operations

```
CLI / SHELL
git status
git add .
git commit -m "Update data pipeline"
git push
```

This example assumes you are in a configured Git repository. It commits and pushes source changes; it does not itself deploy Databricks resources.

Code example: CI/CD pipeline with tests and promotion

```yaml
# .github/workflows/deploy.yml
name: ci-cd
on:
  pull request: PR: validate and test only
  push:
    branches: [main]          # merge to main: deploy

jobs:
  test:
    runs-on: ubuntu-latest
    Steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.11”}
      - run: pip install -e “. [dev]" && pytest tests/unit -q
      - uses: databricks/setup-cli@main
      - run: databricks bundle validate -t staging
        env:
          DATABRICKS_HOST: ${{vars.DATABRICKS_HOST }}
          DATABRICKS_CLIENT_ID: ${{ secrets.SP_CLIENT_ID}}       # service
principal, not a person
          DATABRICKS_CLIENT_SECRET: ${{secrets.SP_CLIENT_SECRET }}

  deploy-prod:
    if: github.ref == 'refs/heads/main'
    needs: test
    runs-on: ubuntu-latest
    environment: production                                       # manual
approval gate
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
      - run: databricks bundle deploy -t prod
        env:
          DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
          DATABRICKS_CLIENT_ID: ${{ secrets.SP_CLIENT_ID }}
          DATABRICKS_CLIENT_SECRET: ${{ secrets.SP_CLIENT_SECRET }}
```

Common mistakes

- Treating Git sync as deployment.
- Deploying untested changes directly to production.
- Letting manual UI edits drift away from source-controlled configuration.
- Forgetting that secrets and environment-specific configuration need safe handling.

[EXAM TIP] A Git push is not a deployment. Pull request → test and `bundle validate`; merge → `bundle deploy` to staging/production with a service principal. Git folders are for developing; bundles are for deploying.

[PRODUCTION BEST PRACTICE] Protect `main`, require passing tests before merge, deploy production only from CI, use short-lived OAuth credentials or workload identity federation instead of long-lived personal access tokens, and keep secrets in the CI secret store and Databricks secret scopes.

Section 9 — Data Modeling (5%)

The new guide has two objectives here: scalable Delta/Iceberg table layouts, and dimensional models using materialized views and Unity Catalog **Metric Views**. Do not study only the older guide's generic dimensional-modeling objective.

9.1 Design scalable Delta/Iceberg table layouts

Official docs: Liquid Clustering · Predictive Optimization · Managed tables

Core concept

A table layout should reflect the data's grain and access patterns while avoiding unnecessary file fragmentation. The guide calls out partitioning, clustering, and balanced file sizes through compaction.

How it works

- Grain: define what one row represents before designing the table.
- Partitioning: physically separates data by partition columns; poor choices can create too many small partitions or oversized partitions.
- Liquid Clustering: organizes data around selected columns and can adapt as query patterns change.
- Compaction/optimization: helps improve file layout and reduce inefficient small-file reads.
- Delta and Iceberg: both are table formats, but the available optimization features and requirements can differ by format and runtime.

When to use it vs. alternatives

- Use clustering when queries repeatedly filter or join on useful clustering columns and access patterns warrant it.
- Consider partitioning only when the data and query patterns justify a partition-based layout.
- Use managed tables and supported automatic optimization where you want Databricks to handle more maintenance.
- Choose Delta or Iceberg based on interoperability, governance, workload and the specific feature requirements; do not assume every feature behaves identically across both formats.

Aspect	Delta Lake table	Iceberg table (in Unity Catalog)
Best when	Databricks-first workloads and the widest Databricks feature coverage	Interoperability with external Iceberg engines is a primary requirement
Optimization features	Liquid Clustering, deletion vectors, Predictive Optimization	Available features and runtime requirements differ; verify per format
VARIANT support	Yes	v3 supports it; v2 does not
Change tracking	Legacy **Change Data Feed**	Automatic CDF on supported v3 tables

SQL example — define a clustered Delta table from source data

```sql
CREATE TABLE main.analytics.sales_by_customer
USING DELTA
CLUSTER BY (customer_id, order_date)
AS
SELECT
  customer_id,
  order_date,
  SUM(amount) AS total_amount
FROM main.raw.sales
GROUP BY customer_id, order_date;
```

This creates one row per `customer_id` and `order_date` combination. The clustering keys are illustrative; select them based on real query patterns and verify that the table and runtime support the chosen feature.

Code example: file-size management and compaction

```sql
-- Avoid small files at write time and compact them in the background
ALTER TABLE main.analytics.sales_by_customer SET TBLPROPERTIES (
  'delta.autoOptimize.optimizeWrite' = 'true',    -- write fewer, larger files
  'delta.autoOptimize.autoCompact'   = 'true'     -- compact small files after
writes
);

-- Manual compaction / clustering when not relying on Predictive Optimization
OPTIMIZE main.analytics.sales_by_customer;

-- Inspect layout health: numFiles, sizeInBytes, clusteringColumns
DESCRIBE DETAIL main.analytics.sales_by_customer;
```

```sql
-- Managed Iceberg table (check current runtime and feature requirements)
CREATE TABLE main.lake.orders_iceberg (order_id BIGINT, order_date DATE, amount
DECIMAL(12,2))
USING ICEBERG
CLUSTER BY (order_date);
```

Common mistakes

- Designing layout before defining the row grain.
- Partitioning by a high-cardinality identifier and producing many tiny partitions.
- Choosing clustering columns that do not match actual query filters.
- Assuming clustering configuration instantly reorganizes all existing files.
- Assuming all Delta optimization features are available for all Iceberg tables and runtimes.

> **[EXAM TIP]** Define the grain first, then layout. Small-file symptoms → compaction (`OPTIMIZE`, auto compaction, optimized writes, Predictive Optimization). Do not partition by user_id or other high-cardinality columns.

9.2 Dimensional modeling, Materialized Views and Metric Views

Official docs: Materialized Views · Unity Catalog Metric Views · Lakeflow pipeline concepts

Core concept

Dimensional modeling organizes analytical data around measurable business processes and descriptive dimensions. The new guide specifically expects you to understand two different features:

- Materialized Views: store precomputed query results and refresh them, potentially incrementally, so consumers avoid repeating expensive transformations.
- Unity Catalog Metric Views: centralize governed business-metric definitions so users can query the same measure across different dimensions without duplicating metric logic.

How it works

- Model the underlying data at a clearly defined grain.
- Use dimensions to describe and filter the business entities, such as date, customer, product or region.
- Use materialized views for precomputed results where refresh latency and storage cost are acceptable.
- Define reusable measures in Metric Views to reduce inconsistent business logic across dashboards and analytics tools.

When to use it vs. alternatives

- Use a regular view when you want a reusable query definition without persisting its result.
- Use a materialized view when repeated expensive queries benefit from precomputed results and its refresh latency is acceptable.
- Use a Metric View when the requirement is a governed, reusable metric definition that supports different dimensions and filters.
- These features can complement each other: a Metric View governs the meaning of a metric, while a materialized view can precompute data for performance. Neither is a universal replacement for the other.

Aspect	Regular view	Materialized view	Metric View
Stores data	No	Yes (precomputed)	No (semantic definition)
Defines	A query	A query and its refreshed result	Measures and dimensions
Performance benefit	None	Yes	None by itself

Aspect	Regular view	Materialized view	Metric View
Consistency of business logic	Per query	Per query	Centrally governed
Typical consumer	Any SQL user	Dashboards with repeated heavy queries	BI tools and analysts across many dimensions

SQL example — build a pre-aggregated sales table that a dashboard can query

```sql
CREATE OR REPLACE TABLE main.analytics.monthly_sales
USING DELTA
AS
SELECT
  DATE_TRUNC('MONTH', order_date) AS sales_month,
  region,
  SUM(amount) AS total_sales
FROM main.raw.sales
GROUP BY DATE_TRUNC('MONTH', order_date), region;
```

This is a pre-aggregation example, not the syntax for creating a Metric View. It assumes `main.raw.sales` has `order_date`, `region` and `amount`. To create the reusable governed metric itself, follow the official Unity Catalog Metric Views documentation, which documents its supported definition and query syntax.

Code example: star schema, materialized view, and a Metric View

```sql
-- Dimension (SCD2-ready) and fact at a clearly defined grain: one row per order line
CREATE TABLE main.gold.dim_customer (
  customer_sk BIGINT GENERATED ALWAYS AS IDENTITY,      -- surrogate key
  customer_id BIGINT NOT NULL, name STRING, region STRING,
  valid_from TIMESTAMP, valid_to TIMESTAMP, is_current BOOLEAN,
  CONSTRAINT pk_dim_customer PRIMARY KEY (customer_sk));

CREATE TABLE main.gold.fact_sales (
  order_id BIGINT NOT NULL, line_no INT NOT NULL,
  customer_sk BIGINT, order_date DATE, quantity INT, amount DECIMAL(12,2),
  CONSTRAINT pk_fact_sales PRIMARY KEY (order_id, line_no),
  CONSTRAINT fk_customer FOREIGN KEY (customer_sk) REFERENCES
main.gold.dim_customer(customer_sk))
CLUSTER BY (order_date);                                 -- PK/FK are informational (not enforced)

-- Materialized view: precomputed aggregate with managed refresh
CREATE OR REPLACE MATERIALIZED VIEW main.gold.mv_sales_by_region_month
AS SELECT DATE_TRUNC('MONTH', f.order_date) AS sales_month, d.region,
SUM(f.amount) AS total_sales
FROM main.gold.fact_sales f JOIN main.gold.dim_customer d ON f.customer_sk =
d.customer_sk
GROUP BY ALL;

-- Metric View: governed definition of "total_sales" reusable across any dimension
CREATE OR REPLACE VIEW main.gold.sales_metrics
WITH METRICS LANGUAGE YAML AS $$
version: 1.1
source: main.gold.fact_sales
dimensions:
  - name: sales_month
    expr: DATE_TRUNC('MONTH', order_date)
measures:
  - name: total_sales
    expr: SUM(amount)
  - name: order_count
    expr: COUNT(DISTINCT order_id)
$$;

-- Consumers ask for the measure; they never re-implement it
SELECT sales_month, MEASURE(total_sales) AS total_sales
FROM main.gold.sales_metrics
GROUP BY sales_month;
```

The Metric View YAML schema (version, joins, filters) evolves; always check the current Metric Views documentation for the exact definition syntax supported in your workspace.

Common mistakes

- Defining the same metric differently in multiple dashboards.
- Mixing fact-table grain and dimension grain in a way that duplicates measures.
- Expecting a materialized view to refresh in milliseconds; the documented behavior is not intended for every low-latency use case.
- Assuming a Metric View automatically solves every physical query-performance problem.
- Choosing precomputation without considering refresh cost and freshness requirements.

[EXAM TIP] A Metric View defines business semantics (one governed definition of "revenue"); a materialized view precomputes query results (performance). "Same metric must mean the same thing in every dashboard" → Metric View. "Expensive repeated aggregation" → materialized view.

[PRODUCTION BEST PRACTICE] Model facts at the lowest useful grain, keep conformed dimensions shared across facts, precompute heavy aggregates with materialized views on serverless pipelines, and put the organization's headline KPIs in Metric Views owned by a data product team.

Final exam-focused revision

Section	Know how to choose...	Key trap
1. Developing Code	Bundles vs. UI, streaming table vs. materialized view, Structured Streaming vs. declarative pipelines, SCD1 vs. SCD2, If/Else vs. For Each	Non-idempotent `foreachBatch`, missing watermark, wrong `sequence_by`, assuming notebook libraries exist on jobs
2. Data Ingestion	Auto Loader vs. COPY INTO, Lakeflow Connect vs. Federation, D2D vs. D2O vs. Clean Rooms	Federation is not a copy; creating a share is not granting access; source CDC must be enabled first
3. Data Manipulation	Joins, windows, VARIANT, `ai_query`, expectations (keep / drop / fail / quarantine)	Missing VARIANT path returns NULL but a bad cast errors; `expect` keeps rows by default
4. Monitoring and Alerting	System tables vs. UI/API vs. event logs vs. alerts	Historical analysis is not the same as alerting or execution-level debugging
5. Cost & Performance	Managed tables, predictive optimization, deletion vectors, clustering, CDF and query profiles	Diagnose the workload and table pattern before choosing an optimization
6. Security and Compliance	Least privilege, ABAC, masks, privacy transformations and purging	A tag or mask alone does not permanently remove sensitive data
7. Data Governance	Comments, tags and privilege inheritance	Parent-level grants can apply to future child objects
8. Debugging and Deploying	Diagnostic source, repair workflow, bundles and Git CI/CD	A Git push is not a deployment, and retrying is not root-cause correction
9. Data Modeling	Grain, layout, materialized aggregation and governed metrics	A Metric View defines business semantics; a materialized view precomputes query results